

# **2025 《程序设计实践》 课程 大作业项目文档**

## 目录

|       |                           |    |
|-------|---------------------------|----|
| 1     | 前言.....                   | 4  |
| 2     | 需求分析.....                 | 5  |
| 2.1   | 功能需求.....                 | 5  |
| 2.2   | 非功能需求.....                | 5  |
| 3     | 总体设计.....                 | 6  |
| 4     | 详细设计.....                 | 7  |
| 4.1   | DSL 设计模块详细设计（模块 1）.....   | 7  |
| 4.1.1 | 设计选型：特化 YAML 格式.....      | 7  |
| 4.1.2 | 设计参考方案分析.....             | 7  |
| 4.1.3 | 分析与最终设计方案.....            | 9  |
| 4.2   | 核心逻辑模块详细设计（模块 2）.....     | 10 |
| 4.2.1 | DSL 识别逻辑.....             | 10 |
| 4.2.2 | 状态机驱动逻辑.....              | 12 |
| 4.3   | 后端框架详细设计（模块 3）.....       | 14 |
| 4.3.1 | 框架核心定位.....               | 14 |
| 4.3.2 | 无登录场景下的会话管理设计.....        | 14 |
| 4.3.3 | 核心 API 接口与请求响应规范设计.....   | 15 |
| 4.4   | 接口文档.....                 | 16 |
| 4.4.1 | 提交用户消息.....               | 16 |
| 4.4.2 | 初始化会话.....                | 16 |
| 4.4.3 | 销毁会话.....                 | 17 |
| 4.5   | 意图识别模块详细设计（模块 4）.....     | 18 |
| 4.5.1 | 语言模型选择.....               | 18 |
| 4.5.2 | 提示词工程.....                | 18 |
| 4.5.3 | 接口函数实现与模块交互.....          | 20 |
| 4.6   | 前端交互详细设计（模块 5）.....       | 21 |
| 4.6.1 | 组件化设计.....                | 21 |
| 4.6.2 | 交互逻辑设计.....               | 21 |
| 4.6.3 | 技术设计风格.....               | 22 |
| 4.7   | 模块交互与接口示意图.....           | 22 |
| 4.8   | 配置文件说明.....               | 23 |
| 4.9   | 日志说明.....                 | 24 |
| 4.10  | 数据库说明.....                | 24 |
| 5     | 测试报告.....                 | 25 |
| 5.1   | DSL 识别逻辑正确性（模块 2 前半）..... | 25 |
| 5.1.1 | 测试驱动设计说明.....             | 25 |
| 5.1.2 | 自动测试脚本设计说明.....           | 26 |
| 5.1.3 | 测试过程与结果.....              | 27 |
| 5.2   | 状态机驱动逻辑正确性（模块 2 后半）.....  | 30 |
| 5.2.1 | 测试驱动设计说明.....             | 30 |
| 5.2.2 | 测试桩设计说明.....              | 30 |
| 5.2.3 | 测试过程与结果.....              | 31 |
| 5.3   | 后端框架测试（模块 3）.....         | 33 |

|       |                                   |    |
|-------|-----------------------------------|----|
| 5.3.1 | 测试驱动设计说明 .....                    | 33 |
| 5.3.2 | 自动测试脚本设计说明 .....                  | 34 |
| 5.3.3 | 测试过程与结果 .....                     | 34 |
| 5.4   | 前端交互测试（模块 5） .....                | 36 |
| 5.5   | 意图识别模块（模块 4） .....                | 37 |
| 6     | DSL 脚本设计与编写指南 .....               | 40 |
| 6.1   | 脚本词法和语法说明 .....                   | 40 |
| 6.2   | 脚本语义说明 .....                      | 42 |
| 6.3   | 脚本用法说明 .....                      | 43 |
| 6.4   | 脚本范例 .....                        | 43 |
| 7     | AI 辅助编程过程说明 .....                 | 45 |
| 7.1   | 本作业使用的 AI 辅助编程工具说明 .....          | 45 |
| 7.2   | 使用 AI 辅助编程的过程 .....               | 45 |
| 7.2.1 | 记录 1：探索选型阶段 .....                 | 45 |
| 7.2.2 | 记录 2：BS 架构的前后端框架初版 .....          | 46 |
| 7.2.3 | 记录 3：前端代码修复 .....                 | 47 |
| 7.2.4 | 记录 4：文法形式定义 .....                 | 47 |
| 7.2.5 | 记录 5：DSL 读取逻辑自动化测试脚本和测试用例生成 ..... | 47 |
| 7.2.6 | 记录 6：用 AI 复现语法分析器的思路 .....        | 48 |
| 7.2.7 | 记录 7：后端测试脚本 .....                 | 49 |
| 7.3   | 使用 AI 辅助编程的经验教训总结 .....           | 49 |
| 8     | GIT 日志 .....                      | 50 |
| 9     | 总结与展望 .....                       | 52 |

# 1 前言

智能客服系统是产品与用户交互的重要入口。传统客服机器人多采用基于规则的应答逻辑，虽能处理结构化问题，但面对复杂、多样化的用户输入时灵活性不足。随着人工智能技术的快速发展，结合大型语言模型/LLM 的智能客服系统逐渐兴起，通过强大的自然语言理解能力，能够更准确地识别用户意图，从而驱动后续业务流程。

基于上述现实意义，本课程作业聚焦于领域特定语言/DSL 的多业务场景智能客服机器人的设计与实现。

项目的核心目标是设计一种可定制的脚本语言，用于描述客服机器人的应答流程，并通过解释器将其转化为具体的交互行为。同时，结合 LLM 的意图识别能力，将用户非结构化的自然语言输入转化为结构化意图，驱动脚本逻辑的动态执行。该 DSL 应具备良好的可读性与可维护性，支持针对不同业务场景定制应答流程，同时解释器需有效集成 LLM，实现从非结构化用户输入到结构化业务逻辑执行的衔接。

同时，本项目作为程序设计实践课程的重要组成部分，也具有强大的实践意义。项目通过完整的开发周期，针对性地训练学生在程序设计、开发调试、测试验证及工程协作中的核心能力：

①开发能力：学生需自主设计 DSL，并实现对应的解释器，掌握从抽象需求到具体代码的转化能力；同时需处理与大模型 API 的集成（如请求构造、响应解析），熟悉开放技术栈的调用与适配。

②排错与调优能力：在开发过程中，学生需面对语法歧义、解释器逻辑错误、大模型响应不稳定等问题，通过日志调试、单元测试、边界案例验证等方式定位并修复缺陷，积累实际工程中的排错经验。

③测试与验证能力：需设计覆盖语法正确性、意图识别准确性、脚本逻辑执行符合性等维度的测试用例（包括手动测试桩与自动化测试脚本），并通过测试驱动开发或事后测试验证功能完整性，理解完整的开发闭环流程。

④工程规范与文档能力：需通过 Git 进行版本管理，并撰写规范的文档（包括需求分析、设计说明、测试报告、DSL 编写指南等），强化工程文档的严谨性与可读性；若使用大模型辅助开发，还需明确记录提示词设计、交互过程及对输出的利用方式，探索对新技术的利用。

通过本项目的实践，学生将在系统设计、模块划分、接口定义、语言设计、AI 集成及测试验证等方面得到全面锻炼，提升解决实际工程问题的能力。

## 2 需求分析

### 2.1 功能需求

**DSL 脚本语言设计：**设计一套语法规则明确的脚本语言，支持描述智能客服机器人的自动应答逻辑，语义需覆盖客服对话流程控制、应答内容定义等核心场景。

**LLM 意图识别集成：**集成一款开放 API 的大语言模型，支持接收用户自然语言输入，调用 LLM API 完成意图识别，并将识别结果作为脚本解释执行的驱动依据。

**解释器核心功能：**实现 DSL 脚本解释器，支持脚本解析、语法校验、基于 LLM 意图识别结果的逻辑执行，输出对应客服应答内容。

**用户交互：**设计合适的用户交互界面进行输入输出，输入内容包括用户自然语言查询，后台的 DSL 脚本文件，输出内容包括客服应答结果。

**测试支撑功能：**提供测试桩（模拟用户输入、LLM API 返回结果）、测试驱动程序及自动测试脚本，支持对解释器功能、脚本语法、意图识别集成的自动化验证。

### 2.2 非功能需求

**语法规范性：**DSL 脚本语言语法需具备明确性、无歧义性，文档中需准确描述语法规则。

**脚本范例提供：**提供不同业务场景的 DSL 脚本范例，确保范例可被解释器正常执行且表现出差异化应答行为。

**兼容性：**解释器需兼容设计的 DSL 语法规则，可正确解析所有符合规范的脚本范例，无语法兼容错误。

**执行效率：**解释器对单条脚本的解析与执行响应时间应足够小，LLM API 调用响应时间也应足够小。

**可靠性：**可连续执行脚本无崩溃，语法错误脚本可返回明确错误信息，无异常退出。

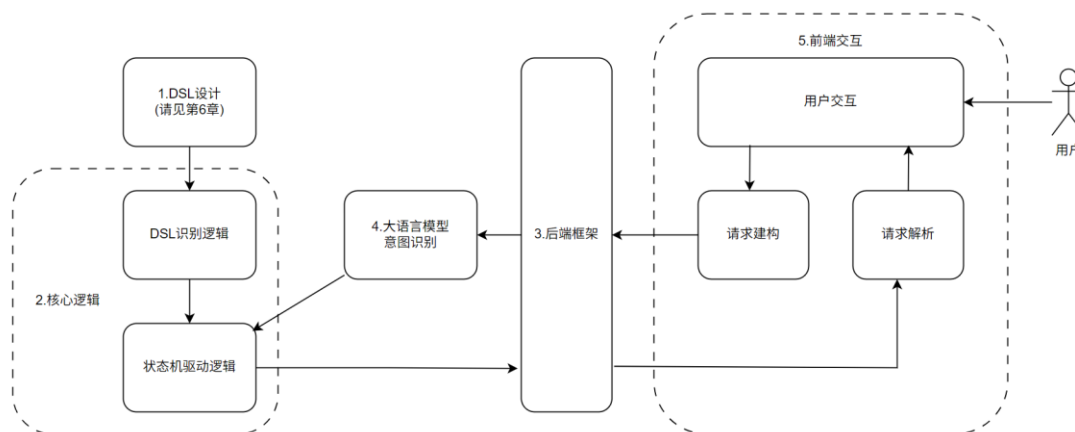
**可测试性：**模块划分需清晰，接口定义规范，支持测试桩接入与自动测试脚本执行，满足测试全覆盖要求。

**版本管理：**项目全程使用 GIT 进行版本管理，每个版本需包含完整、清晰的提交注释。

**开发规范性：**代码编写符合行业规范，模块间接口设计合理，文档撰写清晰，内容完整无冗余。

### 3 总体设计

总体示意图如下，各个模块用数字进行标识：



程序架构上，采用 BS（Browser/Server）模式，实现了前后端分离的现代化 Web 应用设计。前端使用 Vue 3 框架构建单页应用（SPA），同时对核心交互组件化；后端基于 Python 3.12 下的 Django 5.1.7 框架提供 RESTful API 服务。这种架构设计具有良好的可扩展性和维护性，支持并发用户访问。

模块划分上，前端由模块 5 实现。

后端拆解为多个模块：

1.DSL 设计模块。本 DSL 基于 YAML 的规范进一步特化，实现了一个类 YAML 的配置脚本语言，其核心语义与 YAML 有一定区别。

2.核心逻辑模块，主要包括两个逻辑，DSL 的识别逻辑、状态机驱动逻辑两个部分。

DSL 识别逻辑上，可以根据模块 1 的具体实现，采用一个简化的词法、语法分析器。

状态机驱动逻辑，负责根据前一部分的识别结果，以及用户选项，输出对应结果，改变响应状态，这也间接实现了语义分析。

前两个模块共同实现了一个基于规则的客服机器人，采用 Moore 自动机模型。

3.后端框架。这个部分负责把前述功能实现到架构上，提供前端对于逻辑访问的接口，设计数据结构提供并发服务。

4.意图识别模块。分析使用场景和各个语言模型特点，项目决定使用“通义千问-Flash-2025-07-28” 进行完成，需要设计合适的提示词工程。

## 4 详细设计

### 4.1 DSL 设计模块详细设计（模块 1）

#### 4.1.1 设计选型：特化 YAML 格式

本模块的核心目标是设计一套可描述 Moore 自动机模型的 DSL，用于定义智能客服机器人的应答逻辑。在选型过程中，主要基于以下核心需求展开考量：

- ①测试驱动设计与自动化测试便利性：需支持与成熟的读取工具对接，便于验证自研读取程序的正确性，降低测试复杂度；
- ②语法易用性：需参考通用语言的语法范式，降低用户（开发 / 维护人员）的学习成本，实现快速上手编写脚本；
- ③自动机描述适配性：需能精准描述 Moore 机的核心要素，避免冗余语法，确保逻辑描述的简洁性。

基于上述需求，本项目最终决定依托 YAML 格式进行特化设计，而非选择 Mermaid 或其他专用格式。YAML（YAML Ain't Markup Language）是一种基于缩进的轻量级数据序列化语言，其核心特性与本项目需求高度适配：

- ①通用成熟的读取实现：YAML 作为行业通用格式，主流编程语言均提供完善的解析库如 Python 的 PyYAML 就是本项目测试时候使用的解析库，其可用于验证自研 DSL 识别逻辑（模块 2）的正确性；
- ②简洁的嵌套结构：YAML 支持键值对、列表等嵌套数据结构，无需额外定义复杂语法规则；
- ③低学习成本：YAML 语法基于缩进，无多余标记符，熟悉该格式的用户可直接基于已有经验编写 DSL 脚本，降低使用门槛。

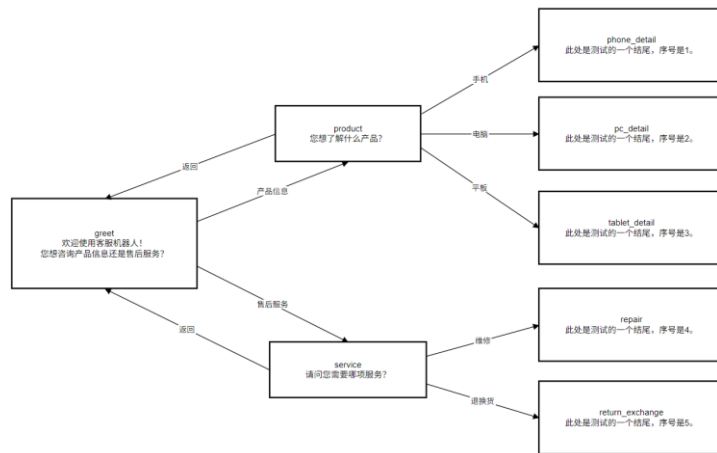
#### 4.1.2 设计参考方案分析

本模块的 DSL 设计思路，主要参考两种成熟的自动机描述方案，结合项目需求进行融合优化。

##### 4.1.2.1 Mermaid 流程图格式

Mermaid 是一种基于文本的图示描述语言，支持通过 flowchart 语法描述状态转移关系，适用于可视化 Moore 机的交互流程。

以下图中的自动机为例：



其 Mermaid 描述如下:

```

flowchart TD
    A["greet<br>欢迎使用客服机器人!<br>您想咨询产品信息还是售后服务?"] -->|产品信息| B["product<br>您想了解什么产品?"]
    A -->|售后服务| C["service<br>请问您需要哪项服务?"]
    B -->|手机| D["phone_detail<br>此处是测试的一个结尾, 序号是 1。"]
    B -->|电脑| E["pc_detail<br>此处是测试的一个结尾, 序号是 2。"]
    B -->|平板| F["tablet_detail<br>此处是测试的一个结尾, 序号是 3。"]
    B -->|返回| A
    C -->|维修| G["repair<br>此处是测试的一个结尾, 序号是 4。"]
    C -->|退换货| H["return_exchange<br>此处是测试的一个结尾, 序号是 5。"]
    C -->|返回| A
  
```

该方案的优势是可视化能力强,可直接生成状态转移图,但存在明显缺陷。

- ①其无法显式描述起始状态、终结状态,需额外通过注释或逻辑推断;
- ②输出函数描述在状态首次出现的位置,不论是编写还是阅读都有诸多不便。
- ③ Mermaid 需兼顾流程图、时序图等多种图示类型,语法存在冗余(如 flowchart TD 声明,多次出现转移函数中相同的待转移状态)。

#### 4.1.2.2 Moore 机形式定义

根据理论计算机的内容,Moore 机的形式化定义为:

$$M = (Q, T, \delta, q_0, F, R, g)$$

其中各要素的含义及与客服场景的映射关系如下表所示:



$Q$ : 状态集合。

$T$ : 输入字母表。此处就是所有用户输入选项的集合。

$\delta$ : 状态转移函数。每一个“当前状态”下的“用户输入选项: 目标状态”实际上就定义了  $\delta: Q \times T \rightarrow Q$ :

$$\delta(\text{当前状态}, \text{用户输入选项}) = \text{目标状态}$$

$q_0$ : 起始状态。

$F$ : 终结状态集。

$R$ : 输出字母表。此处就是所有机器人的回复话语 (output)。

$g$ : 输出函数。每一个“当前状态”下的“output”实际上就定义了  $g: Q \rightarrow R$ :

$$g(\text{当前状态}) = \text{output}$$

这个方法精确，且告知了这个语言必须要定义的内容。然而，让用户定义一句客服机器人的回复语句之前，先去定义  $R$  显然是不现实的。

### 4.1.3 分析与最终设计方案

结合上述两种参考方案的优劣，本模块基于 YAML 格式设计 DSL，核心思路为：保留 Moore 机形式定义的精准性，去除 mermaid 的冗余语法，通过 YAML 的嵌套结构实现要素映射。

项目认为， $q_0$  和  $F$  两个要素需要单独拿出定义，至于  $T$  和  $R$  则不需要显式定义，这是具体场景决定的。和  $Q$  有关的两个要素  $g$  输出函数和  $\delta$  转移函数可以一起定义在一个字段中。这个方案合乎编写逻辑、阅读逻辑，也符合状态机驱动逻辑。

所以，这个 YAML 格式应该含有三个主要字段，于表中后三行：

| 字段名         | 类型       | 是否必填 | 说明          |
|-------------|----------|------|-------------|
| NPC_name    | 字符串      | 是    | 机器人名称，用于标识  |
| start_state | 状态 ID    | 是    | 对话起始状态      |
| end_state   | 状态 ID 列表 | 是    | 标记所有合法的结束状态 |
| states      | 状态结构列表   | 是    | 所有状态的定义     |

第 1 个字段 `start_state` 是一个状态 ID 字符串，定义  $q_0$ ；

第 2 个字段 `end_state` 是一个状态 ID 字符串列表，定义  $F$ ；

第 3 个字段 `states` 是一个列表类的嵌套数据结构，其每一个元素是一个键值对结构。键是状态 ID 字符串（所有键定义了  $Q$ ）；值是另一个键值对结构，其中定义了  $g$  和  $\delta$ 。具体地，值的第一个键值对的键是 `output` 字符关键字，值是  $g$  的输出，第二个键值对的键是 `options` 字符关键字，值是一个键值对结构，每个元素是“用户输入: 状态 ID 字符串”，定义了  $\delta$ 。

另外，语言还应该含有元信息字段，仅设计一个元信息 `NPC_name`，用于标识当前 DSL 脚本对应的客服机器人名称。这个元字段信息的能力是容易扩展的。

DSL 的形式定义、更精确的描述、详细语义对应、范例请见第六章。

## 4.2 核心逻辑模块详细设计（模块 2）

核心逻辑模块是实现智能客服机器人规则驱动能力的核心，主要包含两大核心逻辑：

① DSL 识别逻辑：基于模块 1 的文法，设计词法、语法分析器，读取出其定义的全局配置变量。

② 状态机驱动逻辑：负责根据 DSL 识别逻辑的输出结果及用户输入选项，生成对应应答结果并更新对话状态，间接实现语义分析功能。

模块 2 和模块 1 已经共同构建了一个规则式的客服机器人。

### 4.2.1 DSL 识别逻辑

实现于“yaml\_load.py”文件。DSL 识别逻辑的核心目标是完成对 4.1 节设计的特化 YAML 格式 DSL 脚本的解析，对应编译器的词法分析与语法分析阶段。

本项目的 DSL 分析程序在架构上采用词法分析与语法分析融合的设计方案，该方案属于词法分析的三种经典设计范式之一。

选择此方案的核心原因的是：本 DSL 文法中，除用于表示语法关系的记号（token）外，仅包含 String 一种词法记号。因此，只要在语法分析过程中能明确区分语法级符号与词法级符号，便无需单独设计独立的词法分析程序。结合 Python 语言的特性（如动态类型、字符串处理便捷性），该融合方案的实现难度较低，且可提升解析效率。

语法分析器的设计承接第 1 模块。从编译原理的角度看，模块 1 定义的语言属于 LL(1) 文法，因此本项目的语法分析器采用自顶向下的解析方式，它逐行读取，判断当前行的前缀，根据 DSL 的结构进入下一层解析循环。

例如，以下记号

```
NPC_name:
start_state:
end_state:
states:
状态名必须以：结尾
options 内的元素以 "key: value" 格式出现
- item 代表唯一的列表项
```

这些都能通过看一行的前缀唯一决定下一步逻辑，即，前瞻 1 行即可决定走哪个分支，是 LL(1)文法的典型特征。这体现在：

```

26 # === 3. 找 end_state: 并收集 - 项目 ===
27 end_states = set()
28 while i < len(lines):
29     line = lines[i].strip()
30     i += 1
31     if line == "end_state:":
32         # 进入 end_states 子循环
33         while i < len(lines):
34             sub_line = lines[i].strip()
35             i += 1
36             if sub_line.startswith("- "):
37                 end_states.add(sub_line[2:].strip())
38             else:
39                 # 不是 - 开头, 退出子循环 (不消耗这行)
40                 i -= 1
41                 break
42         break # 跳出外层 while

```

本项目 DSL 文法的另一核心特征是无递归调用关系，整个文法结构呈现树状层级，因此在具体实现时，无需采用传统 LL (1) 分析器常用的递归下降函数，即不需要使用诸如

```

parse_states()
parse_state()
parse_options()

```

等写法，仅通过 while 循环嵌套即可模拟文法的递归下降解析结构。该实现方式与规范的“融合词法分析的 LL (1) 语法分析器”功能完全等价，且因避免了函数递归调用的开销，解析性能更优。这体现在：

```

55 # 新状态: 以冒号结尾
56 if line.endswith(":"):
57     state_name = line.rstrip(":").strip()
58     current_state = {"output": "", "options": {}}
59     states[state_name] = current_state
60
61 # === 子循环: 解析 output ===
62 while i < len(lines):
63     sub_line = lines[i].strip()
64     i += 1
65     if sub_line.startswith("output:"):
66         text = sub_line.split( sep: ":", maxsplit: 1)[1].strip()
67         if text.startswith("'") and text.endswith("'"):
68             text = text[1:-1]
69         current_state["output"] = text
70         break
71
72 # === 子循环: 解析 options ===
73 while i < len(lines):
74     sub_line = lines[i].strip()
75     if sub_line == "options: {}":
76         i += 1
77         current_state["options"] = {}
78         break
79     elif sub_line == "options:":
80         i += 1
81         # 进入 options 键值对子循环
82         while i < len(lines):
83             opt_line = lines[i].strip()
84             i += 1
85             if opt_line.endswith(":"):
86                 # 遇到下一个状态, 退出 options 循环
87                 i -= 1
88                 break
89             elif ":" in opt_line:
90                 key, value = opt_line.split( sep: ":", maxsplit: 1)
91                 current_state["options"][key.strip()] = value.strip()
92                 # 空行或非行忽略
93             break

```

DSL 识别逻辑的输出结果为 DSL 脚本的解析数据，该数据为全局变量且解析完成后保持不可变。为明确数据格式，采用类 Python 变量形式及 TypedDict 类型定义两种方式进行描述。类 Python 变量形式如下：

```
{
    "npc_name": str,
    "start_state": str,
    "end_state": set[str],
    "states": {
        str: {
            "output": str,
            "options": { str: str }
        },
        ...
    }
}
```

TypedDict 严格类型定义描述如下：

```
from typing import TypedDict, Dict, Set

class StateDict(TypedDict):
    output: str
    options: Dict[str, str]

class DSLResult(TypedDict):
    npc_name: str
    start_state: str
    end_state: Set[str]
    states: Dict[str, StateDict]
```

这个全局变量和模块 1 设计的 DSL 的四个基本字段是相对应的。

## 4.2.2 状态机驱动逻辑

实现于“utils.py”文件。

状态机驱动逻辑是基于 Moore 自动机模型实现客服对话流程控制的核心，其核心职责是：根据 DSL 识别逻辑解析得到的状态规则（如状态转移关系、对应输出内容），结合用户输入（经意图识别后），动态更新对话状态并生成应答结果。该逻辑通过模块化函数设计实现，避免与其他逻辑耦合，确保可维护性与可扩展性。

本模块把功能分为对话初始化（init\_session）与对话过程处理（handle\_message）两个函数，核心设计原因如下：

功能解耦合需求：第一次对话和之后的对话交互逻辑是不一致的，拆分便于维护。

适配接口设计：两类函数可直接匹配 RESTful 接口的“会话创建”“消息交互”典型场景——“对话初始化”对应用户发起新对话的接口请求，“对话过程处理”对应用户后续发送消息的接口请求，降低接口开发与逻辑对接的复杂度；

可读性与可维护性：每个函数仅负责单一阶段的逻辑（初始化阶段 / 交互阶段），边界清晰，便于开发人员理解、测试与后续迭代优化。

两个函数设计如下：

1. 对话初始化流程（`init_session` 函数）：该流程负责在用户发起新对话时，完成 Moore 自动机的初始状态配置，返回初始状态的 `output` 和 `options`，为后续交互奠定基础。

2. 对话过程处理流程（`handle_message` 函数）：该流程是状态机驱动的核心，负责处理用户在对话过程中的输入，完成意图匹配、状态更新、应答生成。

状态校验：首先获取当前对话的当前状态，若状态不存在（如未执行初始化流程），则返回无效标识，提示上层补全初始化操作；

终结状态判断：若当前状态属于“终结状态集”（`end_states`），直接返回该状态对应的 `output`（应答内容）与 `options`（可选输入项），并标记“对话结束”，终止后续交互；

意图匹配与状态转移：

接收用户输入（已通过意图识别模块处理为标准化意图 `user_intention`）；

从当前状态的 `options` 配置中，匹配该意图对应的“目标状态”：若匹配成功，更新对话状态为目标状态；若匹配失败（如用户输入无对应选项），则保持当前状态，返回原 `output`（应答内容）并提示用户重新输入有效选项；

新状态处理：

若目标状态属于“终结状态集”，返回目标状态的 `output`（应答内容）与 `options`（可选输入项），标记“对话结束”；

若目标状态为非终结状态，直接提取目标状态的 `output`（新应答内容）与 `options`（新可选输入项），返回给上层调用方，等待用户下一轮输入，继续交互循环。

## 4.3 后端框架详细设计（模块 3）

实现于“urls.py”“utils.py”“views.py”文件中。

后端框架模块基于 Python 3.12 与 Django 5.1.7 框架构建，核心定位是承接前端交互请求、封装核心逻辑模块（模块 2、4）能力、提供标准化 RESTful API 服务，同时可在无登录场景下实现会话状态的有效管理。该模块通过接口与规范设计、会话维护、模块交互及安全保障四大核心能力，支撑用户访问，确保“前端 - 核心逻辑”的解耦与数据流转安全性。

### 4.3.1 框架核心定位

作为整个系统的中间层，后端框架需承担四大职责：

- ①接收前端 Vue 3 单页应用发送的 HTTP 请求，解析请求参数；
- ②调用核心逻辑模块（模块 2）的函数（如 `init_session`、`handle_message`），传递参数并获取处理结果；
- ③封装处理结果为标准化 JSON 响应，返回给前端，确保前端可解析展示；
- ④在无登录场景下，通过 `current_sessions` 维护各网页实例的会话状态，实现多用户访问时的状态隔离。

### 4.3.2 无登录场景下的会话管理设计

考虑到实际应用中多数客服机器人功能无需用户登录即可使用，本项目不实现登录功能，而是通过 `current_sessions`（核心逻辑模块 `utils.py` 中定义的全局字典）实现会话状态维护，确保每个网页实例的对话状态独立不冲突。

`current_sessions` 核心作用：

数据结构：键为 `session_id`（前端生成的会话唯一标识，如基于网页实例 ID + 时间戳生成），值为当前会话的状态（如“greet”“product”等 Moore 机状态）；

维护内容：记录每个会话的必要信息，即当前状态，用户每轮输入后，后端通过 `session_id` 从 `current_sessions` 中获取当前状态，确保对话逻辑连贯。

与后端框架的联动逻辑：

会话初始化（`chat_init` 接口）：调用 `init_session` 函数时，将 `session_id` 与起始状态（如 `start_state` 定义的“greet”）写入 `current_sessions`；

消息处理（`chat` 接口）：调用 `handle_message` 函数时，通过 `session_id` 从 `current_sessions` 读取当前状态，处理完成后更新状态（如从“greet”转为“product”）；

会话销毁（`chat_destroy` 接口）：调用 `remove_session_state` 函数时，从 `current_sessions` 中删除该 `session_id` 对应的记录，释放资源。

并发控制适配：

依托 Django 的多线程处理机制，每个请求通过独立线程调用核心逻辑模块，`current_sessions` 通过 `session_id` 实现会话隔离——不同用户的 `session_id` 唯一，即使并发访问，也只会读写各自对应的状态记录，避免数据冲突；

虽未实现分布式会话(如 Redis 存储),但针对“无需登录的轻量客服场景”,`current_sessions` 的内存级维护已能满足需求,且兼顾响应效率(无需网络 IO)。

### 4.3.3 核心 API 接口与请求响应规范设计

后端框架共设计 3 个核心 API 接口,分别对应“会话初始化”“消息处理”“会话销毁”三大场景,接口实现均位于 `views.py` 文件中,通过 DRF 的 `@api_view(['POST'])` 装饰器限定请求方法为 POST。同时,为确保前后端交互一致性,定义统一的请求与响应规范,各接口的详细参数定义、请求 / 响应格式规范,详见“接口文档”。

会话初始化接口 (`chat_init`):

核心作用: 对应用户首次打开机器人页面或重置对话的场景,初始化 `current_sessions` 中该 `session_id` 的状态,并返回引导信息(如欢迎语、初始可选操作);

关键逻辑: 接收前端传递的 `session_id`,调用 `init_session` 函数完成 `current_sessions` 写入,返回包含引导内容的响应。

消息处理接口 (`chat`):

核心作用: 处理用户每轮输入的消息,基于 `current_sessions` 中的当前状态驱动对话逻辑,返回机器人应答;

关键逻辑: 校验 `message` 非空后,通过 `session_id` 从 `current_sessions` 获取状态,调用 `handle_message` 函数处理输入并更新状态,返回应答结果。

会话销毁接口 (`chat_destroy`):

核心作用: 用户主动结束对话或页面关闭时,清除 `current_sessions` 中该 `session_id` 的记录,释放内存资源;

关键逻辑: 解析 `session_id`,调用 `remove_session_state` 函数删除 `current_sessions` 中的对应记录,返回销毁结果(成功 / 失败)。

请求规范:

所有请求均为 POST 方法,请求体格式为 JSON;

必选参数缺失时(如 `chat` 接口无 `message`),后端直接返回 400 状态码与 `ret=0`,不进入核心逻辑处理;

`session_id` 由前端生成(如基于网页实例唯一标识),作为 `current_sessions` 的键,确保会话唯一性。

响应规范:

所有响应格式均为 JSON,且包含 `ret` 字段(状态标识: 1 = 成功, 0 = 失败);

成功响应 (`ret=1`): 除 `ret` 外,需包含业务字段(如 `output`、`options`、`is_ending`),字段类型固定(`output` 为字符串、`options` 为列表、`is_ending` 为布尔值);

失败响应 (`ret=0`): 基础失败返回 `{"ret": 0}`,复杂失败(如 JSON 解析异常)返回 `{"ret": 0, "msg": "错误信息"}`,便于前端定位问题。

## 4.4 接口文档

为了强调前后端分离架构的开发范式，此处将接口文档单独列出。

### 4.4.1 提交用户消息

接口路径：POST /bot/api/chat/

功能描述：接收用户输入的消息，结合上下文会话，返回模型生成的回复内容及相关控制信息。

请求参数：

| 参数名        | 类型     | 必填 | 描述        |
|------------|--------|----|-----------|
| session_id | string | 是  | 会话唯一标识符   |
| message    | string | 是  | 用户输入的消息内容 |

请求示例：

```
{
  "session_id": "user_123",
  "message": "你好。"
}
```

成功响应：

```
{
  "ret": 1,
  "output": "你好！我是一个智能助手，可以帮你解答问题。",
  "options": ["继续提问", "结束对话"],
  "is_ending": 0
}
```

其中" is\_ending "为 1 表示已经进入结束状态，后端已经关闭会话。

响应失败：

```
{
  "ret": 0
}
```

### 4.4.2 初始化会话

接口路径：POST /bot/api/chat\_init/

功能描述：初始化（或重置）某个会话的上下文状态，通常用于首次加载或重新开始对话时返回引导信息。

请求参数：



| 参数名        | 类型     | 必填 | 描述      |
|------------|--------|----|---------|
| session_id | string | 是  | 会话唯一标识符 |

请求示例:

```
{
  "session_id": "user_123"
}
```

成功响应:

```
{
  "ret": 1,
  "output": "你好！我是一个智能助手，可以帮你解答问题。",
  "options": ["继续提问", "结束对话"],
  "is_ending": 0
}
```

响应失败:

```
{
  "ret": 0
}
```

### 4.4.3 销毁会话

接口路径: POST /bot/api/chat\_destroy/

功能描述: 清除指定会话的上下文状态数据, 通常用于用户退出、会话重置等场景。

请求参数:

| 参数名        | 类型     | 必填 | 描述      |
|------------|--------|----|---------|
| session_id | string | 是  | 会话唯一标识符 |

请求示例:

```
{
  "session_id": "user_123"
}
```

成功响应:

```
{
  "ret": 1
}
```

响应失败:

```
{
```

```
"ret": 0,  
"msg": "错误信息"  
}
```

## 4.5 意图识别模块详细设计（模块 4）

实现于“get\_option.py”文件中。

意图识别模块是连接模块 3 与模块 2 的关键枢纽，核心定位是基于用户输入与当前对话场景，精准匹配预设选项中的目标意图，使得其意图能够应用在基于规则的模型上。

模块提供 get\_user\_intention 接口，接收后端框架（模块 3）传递的 output（场景描述）、options（可选意图）、user\_input（用户输入）参数，调用通义千问 qwen-flash-2025-07-28 模型完成意图识别，最终将标准化结果返回给核心逻辑模块（模块 2），为 handle\_message 函数提供状态切换依据。模块依托精细化提示词工程与性能优化配置，在确保识别准确率的同时实现快速响应，适配客服机器人即时交互需求。

### 4.5.1 语言模型选择

本模块选用通义千问 qwen-flash-2025-07-28 模型作为意图识别核心引擎，理由如下：

①该模型属于通义千问轻量化模型家族，继承 Qwen 系列的长上下文理解能力与多任务适配性，针对即时响应场景进行性能优化，适合短文本意图识别这类轻量级任务；

②客服机器人意图识别为“短期操作意图识别”，需即时性与直接性，模型轻量化架构可快速处理，无需复杂计算；

③支持通过 extra\_body 参数配置推理策略，可直接禁用深度思考流程，进一步缩短响应时间，契合模块快速输出需求；

④模型对“语义匹配”“数字指向解析”“同义替换识别”等场景的适配性强，可覆盖客服意图识别的核心需求。

### 4.5.2 提示词工程

提示词是模块实现意图精准识别的核心，采用“示例学习 + 规则约束 + 动态填充”的三层结构，适配动态对话场景。

#### 4.5.2.1 提示词结构设计

完整提示词由“任务定义区”“示例学习区”“规则约束区”“动态数据区”四部分组成：

①任务定义区：开篇明确“有一个用户意图识别任务”，直接告知模型核心目标，避免理解偏差；

②示例学习区：提供 5 组典型场景示例，包括语义关联型（“欧洲国家”对应“英国”）、反向推导型（“还没有”对应“我现在去...”）、同义替换型（“接受”对应“是”）、数字指

定型（“第二个” 对应 “订单查询”）、无匹配型（无关表述对应特定标识），实现 “以例教模型”；

③规则约束区：强化模型行为规范，明确 “优先在选项里选择，除非实在不合理才返回无合适意图” 的优先级规则，以及 “用户输入数字即对应选项位置” 的特殊处理规则，避免歧义。

前三部分完整展示如下：

有一个用户意图识别任务，接下来是几个你应该学习的示例：

问题：您想去哪个国家？

第 1 个选项是：美国

第 2 个选项是：英国

第 3 个选项是：日本

用户消息：我想去一个欧洲国家

你应该识别为：英国

问题：您完成每日委托了吗？

第 1 个选项是：完成了

第 2 个选项是：我现在去...

用户消息：还没有

你应该识别为：我现在去...

问题：已为您安排行程，您接受该行程吗？

第 1 个选项是：是

第 2 个选项是：否

用户消息：接受

你应该识别为：是

问题：您需要什么服务？

第 1 个选项是：问题咨询

第 2 个选项是：订单查询

用户消息：第二个

你应该识别为：订单查询

问题：您需要什么服务？

第 1 个选项是：问题咨询

第 2 个选项是：订单查询

用户消息：我也不知道，我昨天去了超市

你应该识别为：<本模型认为该用户消息无合适意图>

也就是说，当你认为其中的选项有合理的，就输出它，你应该优先如此认为，即尽量在选项里选择。

除非你认为实在不合理，你可以输出<本模型认为该用户消息无合适意图>，这应该是少数情况。

请注意，用户可能会单纯输入一些数字（或者类似的输入），这意味着他们直接指定想要选择第几个选项，你直接输出即可。

现在，识别下面案例：

④动态数据区：是给出具体问题的部分。将 **output**（场景描述）、带序号的 **options**（如“第 1 个选项是：xxx”）、**user\_input**（用户输入）格式化整合。

### 4.5.2.2 提示词细节优化

选项格式化：为每个选项添加序号（f“第{i+1}个选项是：{option}”），与示例学习区格式保持一致，便于模型关联“数字输入”与选项，降低识别误差；

指令明确性强化：用“也就是说”“请注意”突出核心规则，通过“你应该”“直接输出即可”等祈使句式明确输出要求，减少模糊表述导致的行为偏差；

输出格式暗约束：示例统一采用“你应该识别为：[结果]”格式，结果仅为“选项文本”或特定标识字符串，隐性约束输出格式，无需额外解析多余文本。

### 4.5.3 接口函数实现与模块交互

模块核心能力通过 **get\_user\_intention** 函数实现，该函数作为对外（核心逻辑模块 2）的唯一接口。

```
def get_user_intention(output, options, user_input):  
    """  
    调用远程模型进行用户意图识别。  
    参数：  
        output (str): 问题描述，例如："欢迎使用客服机器人！您想咨询产品信息还是售后服务？"  
        options (List[str]): 选项列表，例如：["产品信息", "售后服务"]  
        user_input (str): 用户输入，例如："我手机坏了"  
    返回：  
        str: 模型识别出的意图，如 "产品信息" 或 "<本模型认为该用户消息无合适意图>"  
    """
```

参数接收：直接接收核心逻辑模块 2 在 **handle\_message** 函数中传递的 **output**、**options**、**user\_input** 参数，无需额外数据转换；

处理执行：对 **options** 进行序号格式化，组装提示词，调用 **qwen-flash** 模型并解析响应；

结果返回：将模型输出的意图结果直接返回给核心逻辑模块 2，模块 2 根据结果判断 Moore 机状态切换方向（匹配则切换状态，无匹配则维持当前状态），完成“模块 4- 模块 2”的交互。

## 4.6 前端交互详细设计（模块 5）

前端交互模块基于 Vue 3 构建，核心定位是为用户提供直观、流畅的客服机器人交互界面，同时承担维护回话状态、与后端交互的关键职责。模块采用组件化设计范式，将客服交互功能封装为独立的 BotService 组件，既确保界面风格统一、交互逻辑清晰，又支持快速集成到任意 Vue 3 项目中，实现“一次开发，多场景复用”。

### 4.6.1 组件化设计

本模块实际上是在实现一个 Vue3 组件，BotService 组件，组件化设计的核心优势体现在独立性、可移植性等等。

**功能独立性：**BotService 组件内置完整的客服交互能力，包括悬浮按钮、聊天窗口、消息展示、输入发送、会话重置等功能，无需依赖外部组件或全局状态，可独立运行。这也实现了样式隔离，避免与项目原有样式冲突，同时支持通过内联样式与外部样式结合的方式，灵活调整界面风格（如悬浮按钮颜色、聊天窗口布局），适配不同项目的视觉设计规范。

**集成便捷性：**在任意 Vue 3 项目中，仅需通过

```
import BotService from './components/BotService.vue'
```

导入组件，并在模板中引用<BotService />，即可快速集成客服功能，无需修改项目原有架构，适配 轻量集成需求；

### 4.6.2 交互逻辑设计

模块的交互逻辑设计以自然、简洁、贴合其他产品前端范式为核心，围绕会话生命周期构建完整流程，同时通过细节优化提升交互流畅度。

**会话生命周期管理：**前端是 current\_sessions 会话状态维护的关键参与方，从会话创建到销毁全程参与。

**会话初始化：**用户点击悬浮客服按钮时，若为首次打开，前端自动调用 initializeChat 函数，通过 axios.post 请求后端 /bot/api/chat\_init/ 接口，传递前端生成的唯一 sessionId（格式为 user\_+随机字符串），触发后端 current\_sessions 的初始化；

**会话交互：**用户输入消息并发送时，前端调用 sendMessage 函数，先将用户消息添加到界面展示，再通过 fetchBotReply 请求后端 /bot/api/chat/ 接口，携带 sessionId 与用户输入，确保后端能精准匹配 current\_sessions 中的对应会话状态；

**会话销毁：**用户关闭浏览器 / 页面时，前端通过 beforeunload 事件监听，调用 navigator.sendBeacon 发送 /bot/api/chat\_destroy/ 请求，传递 sessionId 触发后端 current\_sessions 的记录删除，避免无效会话残留；此外，用户点击聊天窗口“重置”按钮时，前端也会清空本地消息列表并重新初始化会话，同步更新后端 current\_sessions 状态。

**交互细节优化：**

**消息展示：**效仿市面常见即时通讯软件的设计，机器人消息（isBot: true）左对齐、蓝色

系背景，用户消息（isBot: false）右对齐、白色背景，符合主流即时通讯软件的用户习惯；同时通过 `white-space: pre-wrap` 支持消息换行，`max-width: 70%`避免长文本溢出，提升可读性；

选项引导：后端返回 `options` 时，前端通过 `formatOutput` 函数自动将选项格式化为“序号 + 选项文本”的列表（如“1. 产品信息”），附加到机器人回复后，引导用户快速选择，降低输入成本，是一个软性引导设计；

状态反馈：对话结束（`isChatEnded: true`）时，前端自动禁用输入框与发送按钮，避免无效输入；消息发送后自动调用 `scrollToBottom` 函数，通过 `nextTick` 确保滚动到最新消息，无需用户手动滑动。

### 4.6.3 技术设计风格

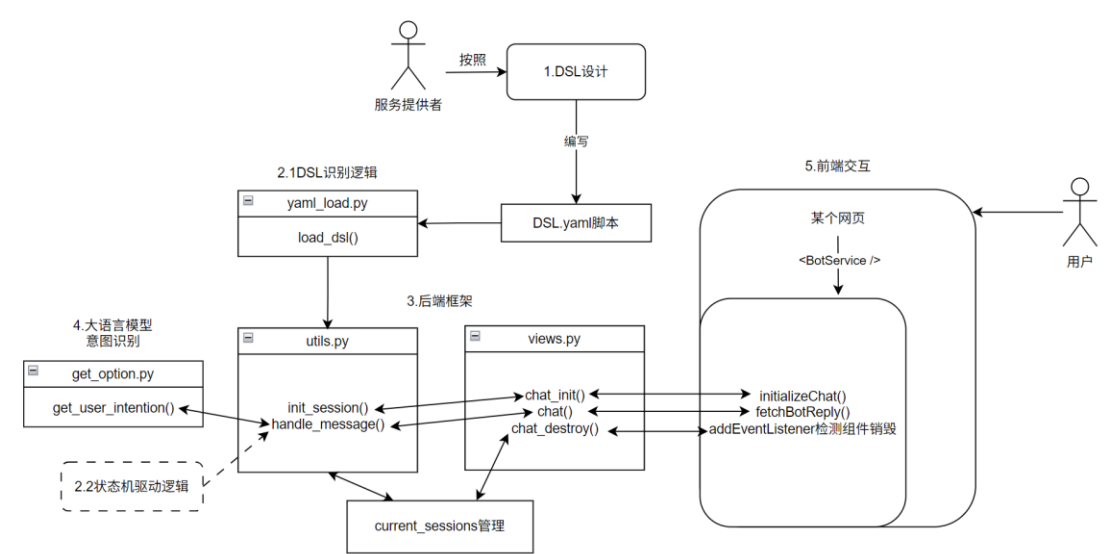
模块采用轻量、高效、敏捷的技术设计风格，贴合 `Vue 3` 的开发范式，同时确保性能与稳定性：

响应式数据管理：使用 `Vue 3` 的 `ref` 函数管理核心状态（如 `isOpen` 控制聊天窗口显示与隐藏、`messages` 存储消息列表、`isChatEnded` 标记对话状态），无需引入 `Vuex/Pinia` 等状态管理库，简化逻辑的同时降低性能开销；

API 请求适配：通过 `axios` 封装后端 API 请求，结合 `config.js` 中的 `getApiUrl` 函数统一管理接口地址，便于环境切换（如开发 / 生产环境 API 地址差异）；同时在请求失败时（如网络异常、后端报错），前端会自动展示友好提示（如“抱歉，机器人暂时无法响应”），提升用户体验；

性能优化：采用 `navigator.sendBeacon` 发送会话销毁请求，确保页面卸载时请求仍能正常发送，避免传统 `axios.post` 在页面关闭时被中断的问题；聊天窗口消息区域使用 `overflow-y: auto` 实现滚动加载，仅渲染当前可视区域消息，避免大量消息导致的界面卡顿。

### 4.7 模块交互与接口示意图



## 4.8 配置文件说明

本 DSL 脚本可以看做一个配置文件。

后端 config 目录下的 settings.py 文件有很多配置项。例如，开发时经常如下配置：

```
# 是否开启调试模式
DEBUG = True
# 允许的域名
ALLOWED_HOSTS = ['*']
# 安装的 Django 应用，其中关于跨域流量的设置是开发时常设置的
INSTALLED_APPS =[
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'core.apps.CoreConfig',
    'rest_framework',
    'corsheaders'
]
# 中间件
MIDDLEWARE = []
# 数据库，默认是 SQLite3，这个配置是因为 Django 采用 ORM 技术
DATABASES = {}
# 开放的静态文件访问
STATIC_URL = []
# 安全密钥
SECRET_KEY
```

前端中，本项目设计了 config.js 配置文件：

```
// API 配置
export const API_CONFIG = {
  BASE_URL: 'http://127.0.0.1:8000',
  ENDPOINTS: {
    CHAT: '/bot/api/chat/'
  }
}

// 获取完整 API URL
export const getApiUrl = (endpoint) => {
  return `${API_CONFIG.BASE_URL}${endpoint}`
}
```

这里配置后端域名（或者 IP 地址）以及端口，并设计了 `getApiUrl` 函数方便前端获取到后端 API 接口的地址。组件实现时，使用 `import { getApiUrl } from '../config.js'` 语句进行导入。

## 4.9 日志说明

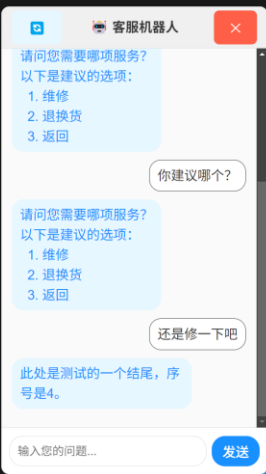
后端控制台会默认记录所有接口请求的情况。

另外，设计 `current_sessions` 的创建、维护、销毁的时候，经常需要确认问题出现在哪个环节上，这时候，后端对于 `current_sessions` 变化情况的记录十分重要，这部分日志示例如下：

```
[15/Nov/2025 21:08:53] "POST /bot/api/chat_init/ HTTP/1.1" 200 270
[15/Nov/2025 21:08:55] "OPTIONS /bot/api/chat/ HTTP/1.1" 200 0
[15/Nov/2025 21:08:56] "POST /bot/api/chat/ HTTP/1.1" 200 266
{'user_0g8tp0mvj': 'intro1'}
[15/Nov/2025 21:09:01] "POST /bot/api/chat/ HTTP/1.1" 200 259
[15/Nov/2025 21:09:18] "POST /bot/api/chat_init/ HTTP/1.1" 200 270
[15/Nov/2025 21:09:21] "POST /bot/api/chat/ HTTP/1.1" 200 266
{'user_0g8tp0mvj': 'intro1', 'user_o42rfo9og': 'intro1'}
[15/Nov/2025 21:09:33] "POST /bot/api/chat/ HTTP/1.1" 200 259
```

另外，后端还会记录大模型识别出的意图：

```
[15/Nov/2025 23:52:40] "POST /bot/api/chat/ HTTP/1.1" 200 142
识别出的用户意图是：<本模型认为该用户消息无合适意图>
[15/Nov/2025 23:52:52] "POST /bot/api/chat/ HTTP/1.1" 200 142
识别出的用户意图是：维修
{'user_mv6fwx07t': 'repair'}
[15/Nov/2025 23:52:59] "POST /bot/api/chat/ HTTP/1.1" 200 94
E:\Python\T5NPCbot\NPCbot\core\utils.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...
E:\Python\T5NPCbot\NPCbot>python manage.py runserver 0.0.0.0:8000
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
November 16, 2025 - 00:01:19
Django version 5.1.7, using settings 'config.settings'
Starting development server at http://0.0.0.0:8000/
Quit the server with CTRL-BREAK.
user_mv6fwx07t <class 'str'>
[16/Nov/2025 00:01:20] "POST /bot/api/chat_destroy/ HTTP/1.1" 200 9
[16/Nov/2025 00:01:21] "POST /bot/api/chat_init/ HTTP/1.1" 200 176
识别出user_josdyohl的用户意图是：售后服务
{'user_josdyohl': 'service'}
[16/Nov/2025 00:01:26] "POST /bot/api/chat/ HTTP/1.1" 200 142
识别出user_josdyohl的用户意图是：<本模型认为该用户消息无合适意图>
[16/Nov/2025 00:01:35] "POST /bot/api/chat/ HTTP/1.1" 200 142
识别出user_josdyohl的用户意图是：维修
{'user_josdyohl': 'repair'}
[16/Nov/2025 00:01:40] "POST /bot/api/chat/ HTTP/1.1" 200 94
```



## 4.10 数据库说明

本项目目前不使用数据库。

然而，项目如果实际投入使用，有一处应该使用数据库进行改造：本项目不实现登录功能，因为确有实际运行的很多网站提供一个机器人功能，且该功能无需登录。所以，对于每一个网页实例，后端都应该进行记录，记录内容一般包括 `sessionID` 以及该回话必要的信息，这由 `current_sessions` 进行实现，详见前文。这种需求使得 `current_sessions` 可以容易地接入内存键值对数据库 `Redis` 进行实现。



## 5 测试报告

### 5.1 DSL 识别逻辑正确性（模块 2 前半）

本部分描述针对项目中用于解析客服对话逻辑的 DSL（npc\_dsl.yaml）所编写的识别、加载与解析脚本的正确性验证工作，即模块 2 中的前半部分工作。包括测试驱动设计、自动化测试脚本设计、测试过程与结果。

#### 5.1.1 测试驱动设计说明

在开发用于解析 npc\_dsl.yaml 的脚本前，为保证 DSL 配置能够被正确识别、加载和转化为程序内部的状态机结构，本项目采用了测试驱动设计的思想，即在编写实际解析逻辑前，首先明确期望的输入输出行为，并针对关键函数设计了相应的测试用例。

在开发 load\_dsl 函数时，预先设想了该函数应能正确读取并解析指定路径的 YAML 文件，返回一个包含 states、start\_state、end\_state 等关键字段的字典结构，随后依据这一预期设计了测试用例，验证函数在给定标准 YAML 文件时是否能正确返回预期数据结构。

预期结果不是人工设计的，而是使用外部库 PyYAML，该库中的函数对于 DSL 的识别几乎和本测试认为的预期结果一致。唯一的区别是，本项目读入 DSL，所有的基本数据类型都是字符串，不会出现数字类型，而 YAML 标准定义（与 PyYAML 的实现上）会把不带引号的数字识别为数字类型。为了修正这一点，使用如下方法得到标准预期结果：

```
def convert_numbers_to_strings(data):
    if isinstance(data, dict):
        return {convert_numbers_to_strings(k): convert_numbers_to_strings(v) for k,
v in data.items()}
    elif isinstance(data, list):
        return [convert_numbers_to_strings(item) for item in data]
    elif isinstance(data, tuple):
        return tuple(convert_numbers_to_strings(item) for item in data)
    elif isinstance(data, (int, float)):
        return str(data)
    else:
        # str, bool, None, 自定义对象等，保持不变
        return data

def load_dsl_yaml_standard(filename):
    with open(filename, "r", encoding="utf-8") as f:
        dsl_data = yaml.safe_load(f)
    dsl_data = convert_numbers_to_strings(dsl_data)
    return dsl_data
```

这样就得到了标准的预期结果。测试时，应对比 load\_dsl 函数和该标准函数的两个输出

是否完全一致。

这种以测试用例指导功能实现的方式，帮助在开发早期发现并修复了若干问题，辅助确定该函数最终使用了一个类 LL(1)解析器。

### 5.1.2 自动测试脚本设计说明

test\_npc\_dsl.py 文件中，有如下核心逻辑：

```
def run_tests(test_dir="test_cases"):
    yaml_files = [f for f in os.listdir(test_dir) if f.endswith(".yaml") or
f.endswith(".yml")]

    passed = 0
    failed = 0
    print(f"开始测试 {len(yaml_files)} 个 YAML 文件...\n")
    for yaml_file in sorted(yaml_files):
        file_path = os.path.join(test_dir, yaml_file)
        print(f"正在测试: {yaml_file}")
        try:
            # 标准方法
            std_result = load_dsl_yaml_standard(file_path)
            # 手动方法
            manual_result = load_dsl(file_path)

            # 深度对比
            diff = deep_compare(std_result, manual_result)
            if diff is None:
                print(f" 成功: 手动解析逻辑对于 '{yaml_file}' 表现正确。")
                passed += 1
            else:
                print(f" 失败: 手动解析与标准解析不一致!")
                print(f"    差异: {diff}")
                print("    标准解析结果:")
                pprint(std_result, indent=4, width=120, compact=False)
                print("    手动解析结果:")
                pprint(manual_result, indent=4, width=120, compact=False)
                failed += 1
        except Exception as e:
            print(f" 异常: 处理文件时出错: {e}")
            failed += 1

    print("-" * 60)

    # 汇总
    print(f"\n 测试完成! 总计: {len(yaml_files)}, 通过: {passed}, 失败: {failed}")
```

该函数读取文件夹下的所有 DSL 测试用例，并且使用 `load_dsl` 和标准方法两种方法得到待测结果和预期结果，使用 `deep_compare` 函数对比两个结果的差异。

`deep_compare` 函数是另一个自行设计的函数，可以用来对比嵌套数据结构的一致性，并在不一致的时候返回不一致的地方：

```
def deep_compare(a, b, path=""):
    if type(a) != type(b):
        return f"[类型不一致] {path}: {type(a)} vs {type(b)}"

    if isinstance(a, dict):
        if set(a.keys()) != set(b.keys()):
            return f"[键集合不同] {path}: {sorted(a.keys())} vs {sorted(b.keys())}"
        for key in a:
            err = deep_compare(a[key], b[key], f"{path}.{key}")
            if err:
                return err
        return None

    elif isinstance(a, (set, list, tuple)):
        a_sorted = sorted(a)
        b_sorted = sorted(b)
        if a_sorted != b_sorted:
            return f"[集合/列表内容不同] {path}: {a_sorted} vs {b_sorted}"
        return None

    elif a != b:
        return f"[值不同] {path}: {repr(a)} vs {repr(b)}"

    return None
```

### 5.1.3 测试过程与结果

在一个历史版本中，对于 6 章中提到的实例，该自动测试程序会返回如下结果：

```

开始测试 1 个 YAML 文件...

正在测试: npc_dsl.yaml1
失败: 手动解析与标准解析不一致!
差异: [键集合不同] .states: ['default_end', 'greet', 'pc_detail', 'phone_detail', 'product', 'repair', 'return_exchange', 'service', 'tablet_
['default_end', 'greet', 'options', 'pc_detail', 'phone_detail', 'product', 'repair', 'return_exchange', 'service', 'tablet_detail']
标准解析结果:
{
  'end_states': {'default_end', 'return_exchange', 'tablet_detail', 'pc_detail', 'phone_detail', 'repair'},
  'npc_name': '产品客服机器人',
  'start_state': 'greet',
  'states': {
    'default_end': {'options': {}, 'output': '对话已结束。感谢您的使用!'},
    'greet': {'options': {'产品信息': 'product', '售后服务': 'service'}, 'output': '欢迎使用客服机器人! 您想咨询产品信息还是售后服务?'},
    'pc_detail': {'options': {}, 'output': '此处是测试的一个结尾。序号是2。'},
    'phone_detail': {'options': {}, 'output': '此处是测试的一个结尾。序号是1。'},
    'product': {
      'options': {
        '平板': 'tablet_detail',
        '手机': 'phone_detail',
        '电脑': 'pc_detail',
        '返回': 'greet'},
      'output': '您想了解什么产品?'},
    'repair': {'options': {}, 'output': '此处是测试的一个结尾。序号是4。'},
    'return_exchange': {'options': {}, 'output': '此处是测试的一个结尾。序号是5。'},
    'service': {
      'options': {'维修': 'repair', '返回': 'greet', '退换货': 'return_exchange'},
      'output': '请问您需要哪项服务?'},
    'tablet_detail': {'options': {}, 'output': '此处是测试的一个结尾。序号是3。'}}}

手动解析结果:
{
  'end_states': {'default_end', 'return_exchange', 'tablet_detail', 'pc_detail', 'phone_detail', 'repair'},
  'npc_name': '产品客服机器人',
  'start_state': 'greet',
  'states': {
    'default_end': {'options': {}, 'output': '对话已结束。感谢您的使用!'},
    'greet': {'options': {}, 'output': '欢迎使用客服机器人! 您想咨询产品信息还是售后服务?'},
    'options': {'options': {}, 'output': ''},
    'pc_detail': {'options': {}, 'output': '此处是测试的一个结尾。序号是2。'},
    'phone_detail': {'options': {}, 'output': '此处是测试的一个结尾。序号是1。'},
    'product': {'options': {}, 'output': '您想了解什么产品?'},
    'repair': {'options': {}, 'output': '此处是测试的一个结尾。序号是4。'},
    'return_exchange': {'options': {}, 'output': '此处是测试的一个结尾。序号是5。'},
    'service': {'options': {}, 'output': '请问您需要哪项服务?'},
    'tablet_detail': {'options': {}, 'output': '此处是测试的一个结尾。序号是3。'}}}

-----
测试完成! 总计: 1, 通过: 0, 失败: 1

```

这就给开发时候提供了一个方便的调试方法,属于本方法带来的另一个好处。该方法错误识别了 DSL 文件,原因是采取了对每一行完全的特征模式识别,使用标记变量标记读取状态等等方法,导致难以维护。后来,项目采用类 LL(1)文法的识别方法,才在开发用例上通过。

之后,项目使用 AI 生成了四个测试文件,人工编写了另一个测试文件。六个测试文件基本情况描述如下:

npc\_dsl.yaml: 参见第 6 章。

npc\_dsl2.yaml: 仿照某游戏 NPC 编写的一个对话脚本。

test1.yaml: Grok 生成的测试文件,场景是“旅游行程规划”,和 test2.yaml 一起生成。

test2.yaml: Grok 生成的测试文件,场景是“健康饮食顾问”,和 test1.yaml 一起生成。

test3.yaml: Grok 生成的测试文件,没有具体场景,表现为一个复杂的迷宫游戏,和 test4.yaml 一起生成。

test4.yaml: Grok 生成的测试文件,没有具体场景,表现为一个复杂的迷宫游戏,和 test3.yaml 一起生成。后两个复杂的测试文件形如:

```

lift_CGA:
  output: "升降梯上升: 抵达云端平台。(CGA→U)"
  options:
    跳伞离开: path_eta
    后退: alpha_step_CG

tunnel_CGB:
  output: "暗道前行: 听到心跳声。(CGB→T)"
  options:
    跟随心跳: path_theta
    后退: beta_step_CG

keyhole_CSL:
  output: "钥匙孔转动: 墙壁消失, 出现iota路径。(CSL→K)"
  options:
    进入: path_iota
    后退: paint_left_CS

```

六个测试用例全部通过测试：

开始测试 6 个 YAML 文件...

正在测试：npc\_dsl.yaml

成功：手动解析逻辑对于 'npc\_dsl.yaml' 表现正确。

正在测试：npc\_dsl2.yaml

成功：手动解析逻辑对于 'npc\_dsl2.yaml' 表现正确。

正在测试：test1.yaml

成功：手动解析逻辑对于 'test1.yaml' 表现正确。

正在测试：test2.yaml

成功：手动解析逻辑对于 'test2.yaml' 表现正确。

正在测试：test3.yaml

成功：手动解析逻辑对于 'test3.yaml' 表现正确。

正在测试：test4.yaml

成功：手动解析逻辑对于 'test4.yaml' 表现正确。

测试完成！总计：6，通过：6，失败：0

## 5.2 状态机驱动逻辑正确性（模块 2 后半）

本部分描述对项目中状态机驱动逻辑的正确性验证工作，也就是模块 2 中的后半部分工作。本项目实现了一个 Shell 模式测试程序（`shellmode.py`），用于模拟用户与状态机的对话过程，并通过手动测试对话的方式验证状态跳转、输出内容、选项展示与终止条件是否符合预期。

### 5.2.1 测试驱动设计说明

在实现状态机驱动逻辑前，为保证后续后端 API 逻辑的正确性，本项目首先聚焦于状态机本身的行为逻辑，即：给定当前状态与用户输入，系统是否能正确选择下一个状态、返回正确的输出文本与选项、并在到达终止状态时正确退出。

在正式编码前，通过对业务需求的分析，明确了以下关键点，作为后续功能实现与验证的依据：

- 每个状态应包含输出文案（`output`）与可选的选项列表（`options`）；
- 用户输入应能在当前状态的选项中找到匹配项，系统据此跳转至下一状态；
- 若用户输入无匹配选项，系统应保持当前状态，并提示重新输入；
- 当状态被标记为终止状态（`end_state`）时，系统应输出最终文案并结束会话；
- 初始状态由配置中的 `start_state` 指定，状态数据来自 YAML 配置文件。

基于以上需求，在编写 Shell 模式交互工具时，已经将这些规则作为设计前提，并在交互流程中逐一落实。虽然未使用自动化测试，但在设计交互逻辑时，已经通过模拟典型对话行为，不断进行开发过程中的测试。

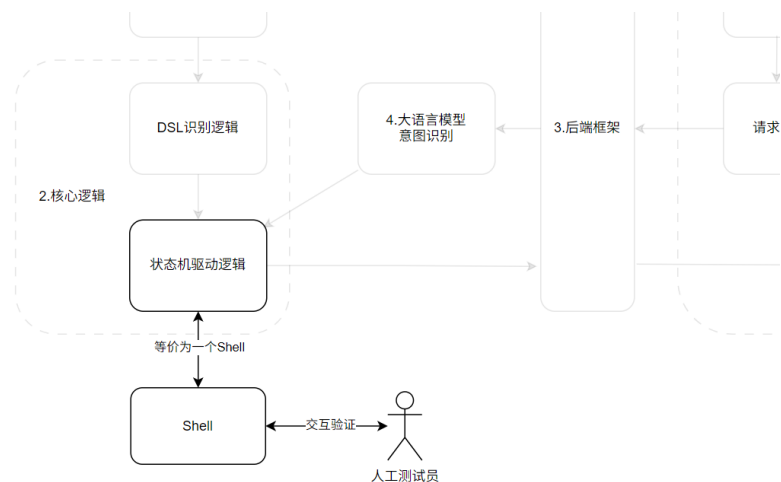
### 5.2.2 测试桩设计说明

该 Shell 程序本质上充当了一个测试桩的角色，它隔离了所有其他模块的依赖，聚焦于核心状态流转逻辑，并通过人工输入输出的方式验证逻辑正确性。具体地：

①它使用 `PYYAML` 库读取 DSL 数据，完全保证前一模块的正确性，并基于此构建内存中的状态机，以及驱动逻辑。

②为避免依赖后端全局变量（如 `current_sessions`）或复杂框架逻辑，该脚本自行维护当前状态（`current_state`），并基于用户输入更新状态，模拟了一个单次会话的生命周期。该脚本未直接复用后端 API 中的状态管理函数（如 `get_session_state` 等），而是重新实现了必要的状态读取与跳转逻辑，但确保其设计思路与后端实现保持一致。

可以认为，这相当于单把这个模块拿出来，进行人工验证测试。具体的测试数据由人工与 Shell 进行交互时构造。



虽然它不是传统意义上的 **Mock** 或 **Stub**，但它通过简化环境、固定输入输出路径，为状态机逻辑提供了可靠的验证基础。

### 5.2.3 测试过程与结果

一次正确而完整的 **Shell** 交互记录如下：

：欢迎使用客服机器人！您想咨询产品信息还是售后服务？  
 您可以输入以下选项：
 

1. 产品信息
2. 售后服务

您想进行什么操作？(输入选项文字，或输入 **quit/exit** 退出) 产品信息  
：您想了解什么产品？  
 您可以输入以下选项：
 

1. 手机
2. 电脑
3. 平板
4. 返回

您想进行什么操作？(输入选项文字，或输入 **quit/exit** 退出) 电脑  
 此处是测试的一个结尾，序号是 2。  
  
 进程已结束，退出代码为 0

当上层输入的返回不合法（例如用户意图被大模型识别为无关，或者大模型工作错误），重新提出交互：

：欢迎使用客服机器人！您想咨询产品信息还是售后服务？

您可以输入以下选项：

1. 产品信息
2. 售后服务

 您想进行什么操作？(输入选项文字，或输入 `quit/exit` 退出) 什么？

 抱歉，未找到匹配的选项，请重新输入。

：欢迎使用客服机器人！您想咨询产品信息还是售后服务？

您可以输入以下选项：

1. 产品信息
2. 售后服务

 您想进行什么操作？(输入选项文字，或输入 `quit/exit` 退出) quit

 感谢使用，再见！

进程已结束，退出代码为 0

更换 DSL 脚本继续进行测试，结果依然如预期描述的那样。



## 5.3 后端框架测试（模块 3）

本部分描述针对后端 API 接口（/bot/api/chat\_init/和 /bot/api/chat/）设计的自动测试脚本，用于验证客服对话系统的核心交互逻辑是否符合预期。由于这个部分的实现基于并发访问控制，设计会话 ID 表的维护，这个部分实际上是在整体测试模块 3。

测试脚本通过模拟用户与系统的多轮对话流程，自动检查接口返回的状态码、字段完整性及业务逻辑正确性。

### 5.3.1 测试驱动设计说明

DSL 脚本采取 npc\_dsl2.yaml，这是模拟某游戏 NPC 机器人人工设计的脚本。

测试目标是覆盖系统中定义的主要对话流程（如经典流程、直接查等阶、探索派遣等），并验证以下核心功能：

会话初始化接口（chat\_init）能否正确返回初始状态信息；

消息交互接口（chat）能否根据用户输入返回正确的输出文案、选项提示及终止状态标识；

系统能否正确处理非法或未匹配的用户输入（如返回 ret=0 拒绝）；

多会话并发场景下接口的稳定性。

在开发测试脚本前，已根据 npc\_dsl2.yaml 中定义的对话流程（如状态跳转、选项匹配规则、终止条件等），梳理出典型的用户输入路径，并明确每一步的预期行为。测试脚本的断言逻辑（如检查 ret、is\_ending 字段）直接基于这些预期设计，确保测试目标与业务需求一致。

对话流程如下：

```
ROUTE_CLASSIC = [ # 经典流程
    ("你是？", "进入 intro1, 凯瑟琳自我介绍"),
    ("原来如此", "回到 greet"),
    ("冒险家协会是做什么的？", "进入 intro2, 介绍冒险家协会"),
    ("原来如此", "回到 greet"),
    ("领取每日委托", "进入 func1, 询问是否完成"),
    ("完成了", "进入 func11, 拿到奖励"),
    ("查看我的冒险等阶", "试图再进入 func2, 不成立, 应该返回 ret=0"),
]

ROUTE_DIRECT_RANK = [ # 直接查看冒险等阶
    ("查看我的冒险等阶", "进入 func2, 回答冒险等阶"),
]

ROUTE_EXPEDITION1 = [ # 探索派遣（蒙德）
    ("查看我的探索派遣", "进入 func3, 选择地区"),
    ("蒙德", "进入 func3e, 应该拿到探索奖励"),
]
```

```

ROUTE_EXPEDITION2 = [ # 探索派遣（璃月）
    ("查看我的探索派遣", "进入 func3，选择地区"),
    ("璃月", "进入 func3e，应该拿到探索奖励"),
]

ALL_ROUTES = {
    "经典流程": ROUTE_CLASSIC,
    "直接查等阶": ROUTE_DIRECT_RANK,
    "探索派遣(蒙德)": ROUTE_EXPEDITION1,
    "探索派遣(璃月)": ROUTE_EXPEDITION2,
}

```

### 5.3.2 自动测试脚本设计说明

测试脚本“test3.py”采用 Python 编写，核心功能模块包括：

#### 1. 基础配置与接口地址

通过 `BASE_URL` 指定后端服务地址（`http://127.0.0.1:8000`），并通过 `get_api_url` 函数拼接具体接口路径（`/bot/api/chat_init/`和 `/bot/api/chat/`）。

#### 2. 测试路线定义

前文已经列出。

#### 3. 核心断言逻辑

函数 `validate_response` 负责检查接口返回的必填字段（`ret`、`output`），并根据当前步骤是否为终止状态（如进入 `func11` 等明确终止节点），验证 `ret` 和 `is_ending` 的正确性（终止状态需 `ret=1` 且 `is_ending=1`，非终止状态需 `ret=1` 且 `is_ending=0`）。

针对特殊场景（如经典流程的最后一步尝试重复查询等阶），通过关键词匹配（如 `func11`、`func2`）或描述中的关键词，额外验证系统应返回 `ret=0` 拒绝请求。

#### 4. 测试执行与并发控制

函数 `run_test_route` 负责单组对话流程的测试：初始化会话后，依次发送用户输入并验证每一步的返回结果；主函数 `main` 使用 `ThreadPoolExecutor` 并发执行多组测试路线（如同时测试经典流程、探索派遣等），提升测试效率，并通过 `future.result()` 捕获单线程异常，避免整体测试中断。

### 5.3.3 测试过程与结果

测试脚本可直接运行（执行 `python` 脚本文件.py），自动输出每一步的交互详情（如用户输入、Bot 回复、断言结果），并在所有测试完成后汇总执行状态。

这是开头一些运行结果的摘录：

```
[主线程] 启动用户 1/30: Session=user_966f2465, 路线='经典流程'
```

开始测试路线: 经典流程 | Session ID: user\_966f2465[主线程] 启动用户 2/30: Session=user\_bf445d4c, 路线='直接查等阶'

开始测试路线: 直接查等阶 | Session ID: user\_bf445d4c[主线程] 启动用户 3/30: Session=user\_50d7d257, 路线='探索派遣(蒙德)'

开始测试路线: 探索派遣(蒙德) | Session ID: user\_50d7d257  
[主线程] 启动用户 4/30: Session=user\_17c296d8, 路线='探索派遣(璃月)'

开始测试路线: 探索派遣(璃月) | Session ID: user\_17c296d8  
[主线程] 启动用户 5/30: Session=user\_4c326751, 路线='经典流程'

开始测试路线: 经典流程 | Session ID: user\_4c326751  
[主线程] 启动用户 6/30: Session=user\_8f26a3b0, 路线='直接查等阶'

开始测试路线: 直接查等阶 | Session ID: user\_8f26a3b0  
[主线程] 启动用户 7/30: Session=user\_950a8350, 路线='探索派遣(蒙德)'

开始测试路线: 探索派遣(蒙德) | Session ID: user\_950a8350  
[主线程] 启动用户 8/30: Session=user\_2623aeae, 路线='探索派遣(璃月)'

开始测试路线: 探索派遣(璃月) | Session ID: user\_2623aeae  
[主线程] 启动用户 9/30: Session=user\_3c79a376, 路线='经典流程'

开始测试路线: 经典流程 | Session ID: user\_3c79a376  
[主线程] 启动用户 10/30: Session=user\_0efeeddf, 路线='直接查等阶'

这是末尾一些结果的摘录:

[经典流程] 第 7 步 (预期被拒绝): ret=0, 符合预期  
测试路线 [经典流程] 完成

Bot 回复: 这是您的探索奖励, 请收好!  
测试路线 [探索派遣(蒙德)] 完成

Bot 回复: 冒险家协会是为了七国各地的冒险家们所设立的支援组织。为了让新手冒险家们更好地适应冒险生活, 老手冒险家们能够更自由地进行冒险, 协会负责整合起冒险家们口口相传的繁杂信息, 将合适的冒险任务分配到冒险家的手中。

--- 输入 [4/7]: '原来如此' (回到 greet)  
Bot 回复: 向着星辰与深渊! 欢迎来到冒险家协会。

--- 输入 [5/7]: '领取每日委托' (进入 func1, 询问是否完成)

Bot 回复: 您完成每日委托了吗?

--- 输入 [6/7]: '完成了' (进入 func11, 拿到奖励)

Bot 回复: 这是您的奖励, 请收好!

--- 输入 [7/7]: '查看我的冒险等阶' (试图再进入 func2, 不成立, 应该返回 ret=0)

[经典流程] 第 7 步 (预期被拒绝): ret=0, 符合预期

测试路线 [经典流程] 完成

所有用户测试执行完毕!

进程已结束, 退出代码为 0

没有报告 assert 错误。

另外, 后端日志系统会记录 current\_sessions, 这是高峰时段的一张摘录, 记录了自动测试程序模拟出的 sessionID 和对应当时状态:

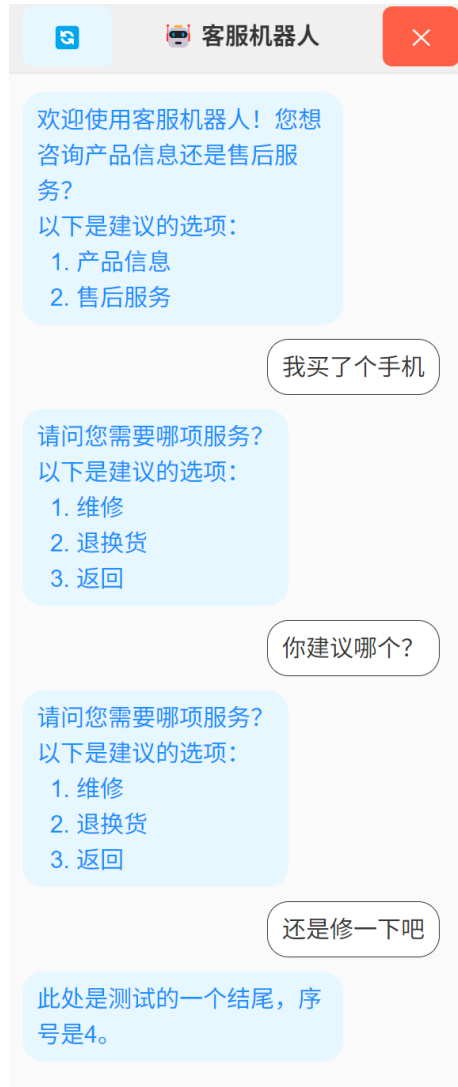
```
{'user_4c326751': 'intro2', 'user_966f2465': 'greet', 'user_3c79a376': 'intro1', 'user_343aca43': 'func3', 'user_2924e5a8': 'intro1', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'greet', 'user_4cb5b568': 'greet'}}
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 87
{'user_4c326751': 'intro2', 'user_966f2465': 'greet', 'user_3c79a376': 'intro1', 'user_343aca43': 'func3e', 'user_2924e5a8': 'intro1', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'greet', 'user_4cb5b568': 'greet'}}
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 259
{'user_4c326751': 'intro2', 'user_966f2465': 'greet', 'user_3c79a376': 'intro1', 'user_2924e5a8': 'intro1', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'greet', 'user_4cb5b568': 'greet'}}
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 373
{'user_4c326751': 'intro2', 'user_966f2465': 'intro2', 'user_3c79a376': 'intro1', 'user_2924e5a8': 'intro1', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'greet', 'user_4cb5b568': 'greet'}}
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 87
{'user_4c326751': 'intro2', 'user_966f2465': 'intro2', 'user_3c79a376': 'intro1', 'user_2924e5a8': 'greet', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'greet', 'user_4cb5b568': 'greet'}}
{'user_4c326751': 'intro2', 'user_966f2465': 'intro2', 'user_3c79a376': 'intro1', 'user_2924e5a8': 'greet', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'func2', 'user_4cb5b568': 'greet'}}
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 373
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 266
{'user_4c326751': 'intro2', 'user_966f2465': 'intro2', 'user_3c79a376': 'intro1', 'user_2924e5a8': 'greet', 'user_6c3664fe': 'intro1', 'user_c3f66c4f': 'greet', 'user_f0290ebc': 'func3', 'user_6c6b9b00': 'greet', 'user_1f53e844': 'greet', 'user_d4812485': 'greet', 'user_clcac91': 'greet', 'user_3a07e963': 'intro1', 'user_00840fd9': 'func2', 'user_4cb5b568': 'intro1'}}
[15/Nov/2025 20:51:41] "POST /bot/api/chat/ HTTP/1.1" 200 259
```

## 5.4 前端交互测试 (模块 5)

本项目未采用前端测试技术, 开发过程中已经通过浏览器实际运行, 模拟了各种可能行为, 修复 BUG 的同时完善前端逻辑。此处无法形成数据化的测试记录。

## 5.5 意图识别模块（模块4）

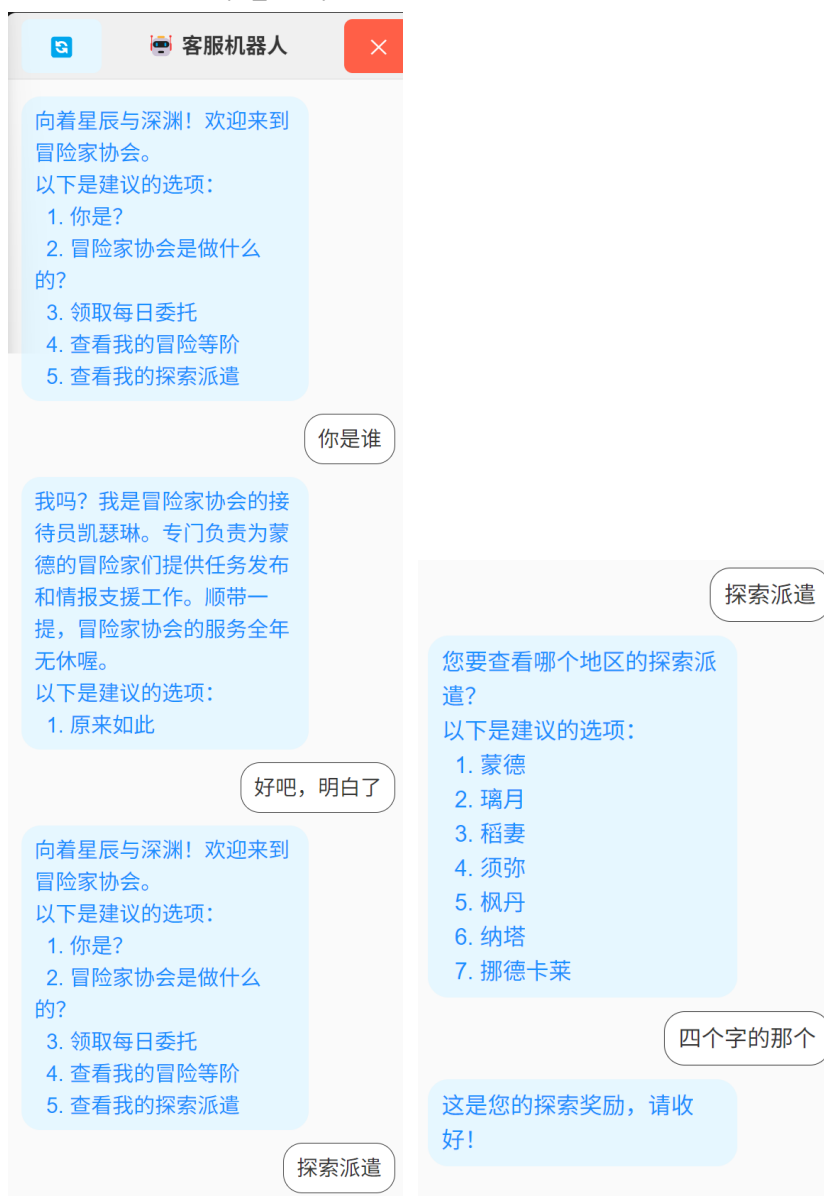
之前的测试已经保证其他模块的正确性，这里直接进行完整配置的测试。使用 `npc_dsl.yaml`，效果如下：



程序的行为是正确的。“我买了个手机”暗示是“售后服务”，“你建议哪个？”没有明显意图，所以模型返回一个特殊的标记，这使得程序返回一个特殊标记，使得程序重新输出刚才话语，“还是修一下吧”暗示了“维修”。

```
user_mv6fwx07t <class 'str'>
[16/Nov/2025 00:01:20] "POST /bot/api/chat_destroy/ HTTP/1.1" 200 9
[16/Nov/2025 00:01:21] "POST /bot/api/chat_init/ HTTP/1.1" 200 176
识别出user_josdyoh1r的用户意图是：售后服务
{'user_josdyoh1r': 'service'}
[16/Nov/2025 00:01:28] "POST /bot/api/chat/ HTTP/1.1" 200 142
识别出user_josdyoh1r的用户意图是：<本模型认为该用户消息无合适意图>
[16/Nov/2025 00:01:35] "POST /bot/api/chat/ HTTP/1.1" 200 142
识别出user_josdyoh1r的用户意图是：维修
{'user_josdyoh1r': 'repair'}
{}
[16/Nov/2025 00:01:40] "POST /bot/api/chat/ HTTP/1.1" 200 94
```

再次测试，使用 npc\_dsl2.yaml 效果如下：



程序的行为在此处看来是正确的。然而，查看后端日志：

```
[16/Nov/2025 00:05:21] "POST /bot/api/chat_init/ HTTP/1.1" 200 270
识别出 user_ikdnisb4b 的用户意图是：你是谁？
{'user_ikdnisb4b': 'intro1'}
[16/Nov/2025 00:05:31] "POST /bot/api/chat/ HTTP/1.1" 200 259
识别出 user_ikdnisb4b 的用户意图是：原来如此
{'user_ikdnisb4b': 'greet'}
[16/Nov/2025 00:05:37] "POST /bot/api/chat/ HTTP/1.1" 200 266
识别出 user_ikdnisb4b 的用户意图是：查看我的探索派遣
{'user_ikdnisb4b': 'func3'}
[16/Nov/2025 00:05:47] "POST /bot/api/chat/ HTTP/1.1" 200 221
识别出 user_ikdnisb4b 的用户意图是：须弥
{'user_ikdnisb4b': 'func3e'}
[16/Nov/2025 00:05:51] "POST /bot/api/chat/ HTTP/1.1" 200 87
```

可以发现“四个字的那个”被语言模型认为是“第四个”的意思，而不是对于选项文字形态“挪德卡莱”的描绘。鉴于这种识别能力常常是通用语言模型的弱项（例如数长度这种  $O(n)$  推理，短的数长度使用 CoT 可以解决，但降低速度，且长的数长度则根本无法解决），且不宜给出语义不一致的硬提示词工程（有些识别涉及到序号，有些涉及到常识推理，有些涉及到数长度等数学推理），该问题本项目不予解决。

最后，使用 test1.yaml 测试，效果如下：



程序行为基本正确。关于“是”或“否”的意图识别出现错误，提示词工程加入类似提示，依然不给出正确结果，除非说出“我要购买”才能给出正确识别：



这可能是语言模型本身的问题，因为另一个语言模型（腾讯元宝，上方右图）在相同的提示词工程下可以正确给出回答。鉴于实用考虑，本项目对该问题亦不作修复。

## 6 DSL 脚本设计与编写指南

### 6.1 脚本词法和语法说明

本 DSL 设计依托于一个特化的 YAML 格式。下面采用 BNF 范式严格定义这个文法，换行符使用"\n"进行表示，换行符可以任意添加数次，非终结符使用尖括号标出：

```
<chatbot> ::=
  NPC_name: <string> \n
  start_state: <state_id> \n
  end_state: <end_state_list> \n
  states: <state_map> \n

<end_state_list> ::=
  \n <indent> - <state_id> <end_state_list> |
  <empty>

<state_map> ::=
  \n <indent> <state_def> <state_map> |
  <empty>

<state_def> ::=
  <state_id> : \n
  <indent> output: <string> \n
  <indent> options: <option_map>

<option_map> ::=
  {} |
  <option_list>
<option_list> ::=
  \n <indent> <option_text> : <next_state_id> <option_list> |
  <empty>

<string> ::= \" <char>* \"
<char> ::= <除了换行符、不正确转义、引号的任意 unicode 字符>

<state_id> ::= <identifier>
<next_state_id> ::= <identifier>
<identifier> ::= [a-zA-Z_][a-zA-Z0-9_]*

<indent> ::= <两个空格>
<empty> ::= <空串>
```



下面更加通俗地描述这个文法。一个完整的该 DSL 包含以下部分：

```
NPC_name: 机器人名字
start_state: 开始的状态 ID
end_state: 哪些状态算对话结束（一般是多个）
states: 所有状态的详细定义
```

其中 `end_state` 下一般是一个列表，有多个项，在 `end_state` 下具有一个缩进，且用“-”和一个空格开头：

```
end_state:
- default_end
- phone_detail
```

`states` 下定义对话的每一状态，每一状态在 `states` 下具有一个缩进。其中，一个状态，需要用状态 ID 进行表示，用一个冒号开辟自己的详细描述，详细描述在状态 ID 下也具有一个缩进，详细描述指的是 `output` 和 `options` 两个信息。形式如下：

```
greet:
  output: "欢迎使用客服机器人！您想咨询产品信息还是售后服务？"
  options:
    产品信息: product
    售后服务: service
```

其中 `options` 也具有自己的内部结构，在 `options` 下具有一个缩进。它的内部结构是一些键值对，键是建议用户输入的相关话语，值是对应的要跳转的状态 ID。

请注意，一般来说，`end_state` 规定的终结状态不具有 `options` 信息，这时候，在“`options:`”的同一行使用花括号进行表示，即使用“`options: {}`”表示。

各个状态建议使用标识符，其应以字母或者下划线开头，之后具有若干字母、数字、下划线。然而，使用数字开头，或者汉字等是可行的，但不建议这样。

`output` 之后的信息不使用引号也是可行的，这和 `YAML` 的定义类似，但不建议这样。不使用缩进，或者使用其他空白符号进行缩进是可行的，但不建议这样。

## 6.2 脚本语义说明

核心是要理解状态结构。文法中，`<state_def>`最复杂，它定义了一个状态，以及这个状态的转移函数，使用它的`<state_id>`做唯一标识。文法展开后形式如下：

```
<state_id>:
  output: "文本内容"
  options:
    用户输入选项 1: 目标状态 1
    用户输入选项 2: 目标状态 2
    ...
```

这样就可以理解这个脚本的语义和用法。本质上，这个类 YAML 语言定义了四个字段：

| 字段名         | 类型       | 是否必填 | 说明          |
|-------------|----------|------|-------------|
| NPC_name    | 字符串      | 是    | 机器人名称，用于标识  |
| start_state | 状态 ID    | 是    | 对话起始状态      |
| end_state   | 状态 ID 列表 | 是    | 标记所有合法的结束状态 |
| states      | 状态结构列表   | 是    | 所有状态的定义     |

其中 NPC\_name 属于一个元信息，和状态机的形式定义无关。之后的字段定义了一个 Moore 机：

$$M = (Q, T, \delta, q_0, F, R, g)$$

这个状态机的要素如下：

$Q$ ：状态集合。正是 states 字段定义的一个个状态的 ID；

$T$ ：输入字母表。此处就是所有用户输入选项的集合；

$\delta$ ：状态转移函数。由 states 字段定义，每一个“当前状态”下的“用户输入选项：目标状态”实际上就定义了 $\delta: Q \times T \rightarrow Q$ ：

$$\delta(\text{当前状态}, \text{用户输入选项}) = \text{目标状态}$$

$q_0$ ：起始状态。由 start\_state 字段定义。

$F$ ：终结状态集。由 end\_state 字段定义。

$R$ ：输出字母表。此处就是所有机器人的回复话语（output）；

$g$ ：输出函数。是由 states 字段定义的，每一个“当前状态”下的“output”实际上就定义了 $g: Q \rightarrow R$ ：

$$g(\text{当前状态}) = \text{output}$$

按照以上语义，用户可以编写一个 Moore 机，作为该机器人的回复逻辑。

另外，关于 YAML 的语法，空字典对象，使用一对空花括号表示，这在表示`<option_map>`是常用的（不允许省略），尤其是描述一个终结状态的时候，其状态转移往往是无意义的。

识别该语言时，应只使用一种基本数据结构，即字符（以及字符串），不应出现数字类型。

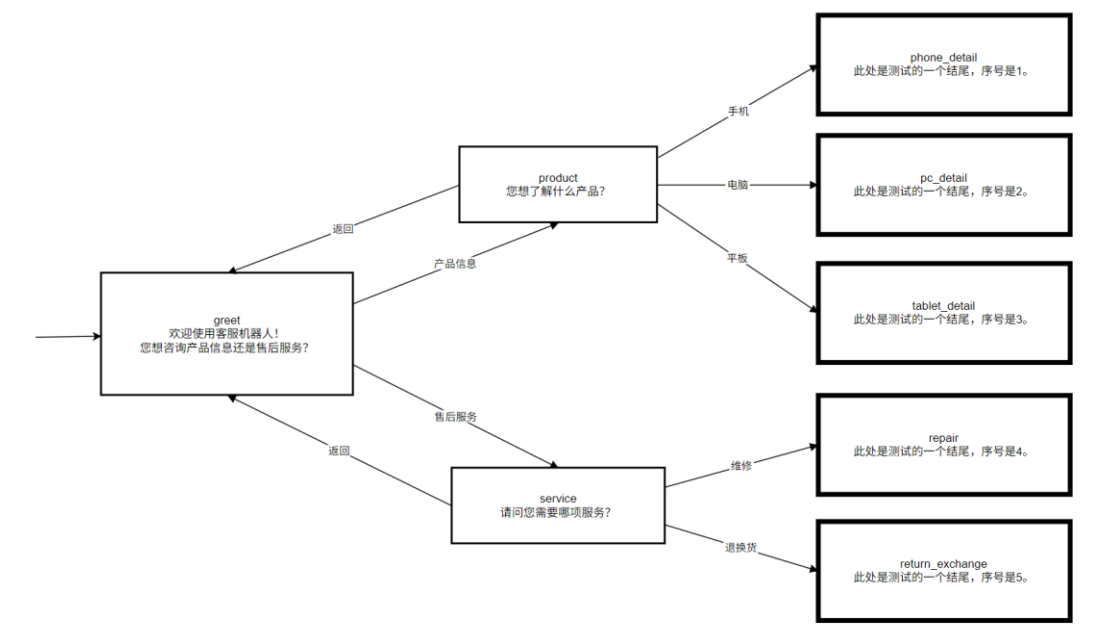
## 6.3 脚本用法说明

为了获得更好的编写体验（如高亮，自动缩进），建议用户使用安装了 YAML 插件的 VS Code，或者自带 YAML 支持的文本编辑器。

如果没有以上工具，用户右击文件，使用记事本打开进行编写也是可行的。

## 6.4 脚本范例

以如下 Moore 自动机为例：



按照以上设计，编写如下代码：

```
NPC_name: 产品客服机器人
start_state: greet
end_state:
  - default_end
  - phone_detail
  - pc_detail
  - tablet_detail
  - repair
  - return_exchange
states:
  greet:
    output: "欢迎使用客服机器人！您想咨询产品信息还是售后服务？"
    options:
      产品信息: product
      售后服务: service
```

```
product:
  output: "您想了解什么产品？"
  options:
    手机: phone_detail
    电脑: pc_detail
    平板: tablet_detail
    返回: greet

phone_detail:
  output: "此处是测试的一个结尾，序号是 1。"
  options: {}
pc_detail:
  output: "此处是测试的一个结尾，序号是 2。"
  options: {}
tablet_detail:
  output: "此处是测试的一个结尾，序号是 3。"
  options: {}

service:
  output: "请问您需要哪项服务？"
  options:
    维修: repair
    退换货: return_exchange
    返回: greet
repair:
  output: "此处是测试的一个结尾，序号是 4。"
  options: {}
return_exchange:
  output: "此处是测试的一个结尾，序号是 5。"
  options: {}

default_end:
  output: "对话已结束，感谢您的使用！"
  options: {}
```

# 7 AI 辅助编程过程说明

## 7.1 本作业使用的 AI 辅助编程工具说明

两个商业 AI，腾讯元宝和 Grok。

腾讯元宝是“基于腾讯混元大模型的 AI 应用”。

Grok 是“由 xAI 设计的免费人工智能助手”。

## 7.2 使用 AI 辅助编程的过程

### 7.2.1 记录 1：探索选型阶段

选型脚本依托的语言、了解编译技术、探索 AI 实现状态机驱动逻辑的可行性。

腾讯元宝。其回复很长，只在必要时候介绍其回答。

第 1 轮：yacc 是什么

第 2 轮：python 怎么使用类似工具

AI：首先介绍“PLY 是 Python 版的 Lex & Yacc，完全用 Python 实现，功能类似于 Unix 下的 Lex 和 Yacc（或 Flex 和 Bison）。”最后提到“✅ 如果你是初学者，想快速上手并体验类似 Yacc 的功能，我推荐你试试 Lark——它比 PLY 更简单、更直观，而且支持 EBNF 语法，非常友好！”

🔧 如果你有兴趣，我也可以给你一个用 Lark 实现的同样功能的计算器例子，或者带你用 Lark 写一个简单的 DSL（领域语言）。欢迎继续提问！”

第 3 轮：用 Lark 写一个简单的 DSL 看看效果

第 4 轮：帮我做一个描述客服机器人运行逻辑的脚本语言，它应该描述了一个有输出的自动机，可以读取成为一个语法树，之后客服机器人会按照这个语法树的逻辑，结合用户输入去运行，状态得有一个超时的设计，就是多少秒之后转移到一个结束状态，还得有一个时限内有输入，但是不是合法状态转移的设计

AI：（使用 Lark 读取刚才的 DSL）最后它说



🔧 想继续？

我可以继续帮你：

- ✅ 实现这个状态机引擎（根据语法树 + 用户输入跑起来）
- ✅ 加入 异步超时控制（asyncio + timer）
- ✅ 支持 正则表达式选项 而不仅是固定关键词
- ✅ 把 DSL 写成 YAML 格式并用 PyYAML 加载（更友好）
- ✅ 支持 变量、上下文、参数传递（比如用户选了“手机”，下一状态显示型号列表）

🔧 如果你想要我把这个 DSL 脚本改成 YAML 格式，或者帮你实现状态机引擎部分，欢迎继续提问！我可以帮助你一步步实现完整的客服机器人逻辑 😊

第 5 轮：把 DSL 脚本改成 YAML 格式有什么好处？PyYAML 好用吗？

AI：（回答上述问题）

第 6 轮，不置可否地继续询问：驱动逻辑怎么写

AI：（默认我采用 PyYAML，并给出了很长的代码）

第 7 轮，由于认为 AI 设计的脚本不好，自行设计后：我重新设计了 yaml：（脚本早期版本内容）现在修改 run\_customer\_service\_fsm 的逻辑，力求简洁易理解

AI：（除了前后的各类文字，中间有一段较长的代码）

结果：

①AI 最终给出的代码作为本项目的 Shell mode 进行使用，其需要经过大量修改。

②AI 帮助项目确定 DSL 基于 YAML 语法去实现，且提供了 PyYAML 这个工具，该工具早期用作更复杂模块开发时 DSL 识别逻辑的代替品，并且提供了测试 DSL 识别逻辑的一个方法。

## 7.2.2 记录 2：BS 架构的前后端框架初版

尝试用 AI 构建前后端框架。腾讯元宝。

第 1 轮：（类似上一个记录的脚本，让它写一份识别逻辑，使得它熟悉核心逻辑，再构建前后端框架代码）

第 2 轮：现在我想把这个运行成一个 BS 项目中的小组件，效果类似于随便一个网页，右下角出现一个客服悬浮按钮，点击之后就打开一个和机器人聊天的悬浮窗口。我已经用 vue3 和 vite 新建了一个项目，也使用 Django 新建了一个项目和一个 app 包，你能帮我实现核心代码吗

AI：（非常长的一段回答）

第 3 轮：有问题，用户第一次点击按钮打开会话框，你应该有一些预处理，例如创建好 sessionID 发向后端某个接口，后端记录下来，告知前端第一句话是什么，你现在打开都没有反应。而且你确定你使用了 POST 正确请求后端？后端的"OPTIONS /bot/api/chat/ HTTP/1.1" 200 163 并不是这样。你现在机器人只会说：抱歉，机器人暂时无法响应，请稍后再试。

AI：（非常长的一段回答）

第 4 轮：现在我想要每次点开会话框，如果是第一次点开，就加载话语，如果是之前回话已经结束，也加载话语。而不是想现在这样，每次加载都会重新说一遍已经说过的话，且不能重新开始对话。为此，后端可能需要使得已经结束的对话回到一开始

第 5 轮：能不能仔细谈谈为什么需要 init 和 message 的区别？用 current\_sessions 里有无会话标识进行区分可以吗

AI：（非常长的一段回答）

结果：代码的雏形是可用的，但是需要人工做大量修改。其中有很多不必要的代码段导致错误。

### 7.2.3 记录 3：前端代码修复

使用 Grok。

前端代码经过大幅度人工修改之后已经可以正常使用。现在，需要增加一个销毁组件时候调用的功能。

开始本计划人工实现，类似网页生命周期那样使用 Vue3 的组件生命周期，然而，这对于直接关闭网页、刷新网页等情况不适用，和 AI 对话很久之后意识到这一点。

这段记录有 9 轮，其中多次因为 AI 代码过于臃肿复杂、不必要的各种复杂排错逻辑等问题让其精简代码。其中又一轮还导致前端销毁组件时发送两次请求。AI 在最后一轮给出正确代码。

### 7.2.4 记录 4：文法形式定义

使用 Grok。

只有一轮：我设计了一个领域特定语言，基于 yaml 文件格式，这是一个例子：（脚本）其实就是描写一个聊天机器人的回复逻辑自动机。按照这个范例，你给我一份这个语言的文法定义，用法说明

AI：以下是针对您设计的领域特定语言（DSL）——一种基于 YAML 的聊天机器人状态机描述语言——的完整文法定义（BNF 范式）和详细用法说明。一、文法定义（BNF 范式）（省略.....）（写完 BNF 之后，AI 还说了大量和原要求无关的话语）

结果：该文法充斥大量错误。但是基本结构是可用的，经过大量修改成为最终采用的文法形式定义。

人工修改的时候，参考了一份 ISO 对于 C99 的定义标准，其中关于文法的定义成为这里解决的风格。

### 7.2.5 记录 5：DSL 读取逻辑自动化测试脚本和测试用例生成

使用 Grok。

该任务比较简单，只有一轮对话：这里有一段代码：第一种绝对正确的读取方法：

```
with open("npc_dsl.yaml", "r", encoding="utf-8") as f:
```

```
    dsl_data = yaml.safe_load(f)....
```

手动实现的读取方法：

```
textdef load_dsl(filename):...
```

现在，请帮我写一个自动测试脚本，读取一个文件夹下的所有 YAML 文件，每个文件调用这两个函数，然后对比结果，完全一致打印一句手动识别逻辑对于某文件表现正确之类的话。

AI：（给出正确代码）

测试用例生成另外开了一段对话。

这个复杂的提示词是因为之前四次效果都不理想，过于简单或者过于复杂，第五次编辑原消息才给出合适结果：

用户：仿照这个格式，生成另外两段文本，用于测试，请注意是为了测试，你的状态机设计应该有一定变化，深度可以大一些，不许只有三层，甚至允许成环，但是不允许任何新功能（诸如循环次数检测、局部变量存储，什么都没有，就是一个状态机而已。然而，你可以通过设计状态模拟这个机器人好像记住了某些局部变量，实际上仍是最简单的自动机）：

（代码示例）

AI：（没有任何其他话语，直接给出了两段代码，也就是本项目附带的 `test1.yaml` 和 `test2.yaml` 文件）

第 2 轮：再编写两个，这两个可以没有具体场景，但是比较复杂，深度和状态数都很高

AI：（没有任何其他话语，直接给出了两段代码，也就是本项目附带的 `test3.yaml` 和 `test4.yaml` 文件）

## 7.2.6 记录 6：用 AI 复现语法分析器的思路

4.2.1 节已经详细说明了该语法在实现上的特殊性，即不必按照 LL(1) 识别器的范式，使用嵌套循环即可等价且高效地完成这个分析器。这里尝试用 AI 实现这个逻辑。为了使得 AI 理解这段逻辑，输入了大量的提示词。

第 1 轮：这里有一段类 YAML 语言：（DSL）然而，我现在需要你手动实现一套加载这个特化 YAML 语言的代码。你应该使用最简单的方法实现这件事，不要使用复杂的编译原理的内容。

例如，你应该一行一行读取内容，跳过空行。

读取到 `NPC_name` 开头，直接使用 `split` 截断这一行，把后面的代码去掉左右空格，存到变量 `npc_name` 里。

然后继续读，读到 `start_state` 开头，直接使用 `split` 截断这一行，把后面的代码去掉左右空格，存到变量 `start_state` 里。

然后继续读，读到 `end_states` 开头，跳过这一行（这一行不会有任何数据）然后修改一个标记变量，使得我们开始读取 `end_states` 列表，接下来的循环，你应该检查除了空行以外的第一个字符是不是小横线，如果是，把之后内容加入到 `end_states` 列表，直到如果不是小横线，跳出这个状态。

然后继续读，读到 `states` 开头，你应该设计两个状态变量，第一个表示我们正在读取某个状态（例如 `greet`）的 `output` 或者 `options`，另一个标记我们正在读取一个 `options`。请注意 `options` 只能是两种情况：一堆花括号，或者是一些键值对。

你不应该做太多的写法检查，因为这个语法非常死，非常固定，你的程序可以且尽可以识别类似这个事例的语言，而不是什么其他的任何一个 YAML 语言。

AI：（给出错误代码）

第 2 轮省略。

第 3 轮：你还没发现你代码的最大问题吗。你试图把每一行都经过相同的处理逻辑，可是事实上根本不要这样，代码应该一遍跑完，用局部循环。

你现在期望识别 `NPC_name` 开头，往后过语句，直到找到这个，跳出循环。



然后你现在期望识别 `start_state` 开头，往后过语句，直到找到这个，跳出循环。  
然后你现在期望识别 `end_states` 开头，往后过语句，直到找到这个，进一个子循环，  
子循环期望所有输入都是小横线开头，不是才能跳出循环。  
然后你现在期望识别 `states` 开头，往后过语句，直到找到这个，进子循环，  
子循环期望一个结尾是冒号的语句，进子循环  
期望 `output` 行，直到，跳出 `output` 行  
期望 `options` 行，直到，跳出 `options` 行，或者进子循环。  
重新编写代码。

AI: (给出正确代码)

第 4 轮: 请认真反思为什么我们之前失败，为什么这个能够成功，尤其是从理论角度说明，关注文法结构（属于什么文法，什么特点）、语句前缀各种理论问题

AI: (详细分析了前两版代码和正确代码的区别。然而，其最后才指出这是一个 LL(1) 分析器，之前的分析从前缀、嵌套歧义、局部子循环等角度展开)

## 7.2.7 记录 7: 后端测试脚本

采用腾讯元宝。没有太特别的提示词设计，一共 4 轮对话，由于该 AI 回答过于长，这里从略。其中，由于 AI 没有完全理解这个模型的应答逻辑，多次指正其才终于理解，给出了正确测试代码。

## 7.3 使用 AI 辅助编程的经验教训总结

总结如下。

- 1、没有思路时，做调研、技术选型，使用 AI 是实用的。
- 2、逻辑简单的代码 AI 通常能够给出好的结果。
- 3、逻辑简单或复杂并不一定和代码量有关。有些函数可能逻辑复杂，但是代码量很小。这时候需要多次对 AI 做出反馈调整；或者，需要极其细致仔细地向 AI 描述已有思路，然而，这依然难以避免反馈调整。
- 4、大代码量的任务，如果要使用 AI，应该做拆解，让 AI 一部分一部分地解决。否则只能生成一个基本框架可能可以利用的代码。
- 5、理论计算机（如自动机描述等）相关的内容涉及到复杂的数学符号，AI 对此可能会出现混乱。
- 6、AI 生成测试用例是非常高效的，只需要进行一定程度的人工检查即可。
- 7、AI 生成前端代码通常更强大。
- 8、通常来说，AI 生成的内容都需要做修改。这个修改除了其考虑上的逻辑错误，还有很多情况是由于其过度考虑带来的额外问题。

## 8 GIT 日志

前端 git log:

```
E:\Trae Project\npcbot-frontend>git log
commit ec039db91764cce485a2619815f1255447462caa (HEAD -> master)
Author: LeoTian <LeoTian163@163.com>
Date: Sat Nov 15 14:14:17 2025 +0800

    关闭会话的内存管理

commit c6397b905ad7a69b280a6271f0eb18e3fc7fe64a
Author: LeoTian <LeoTian163@163.com>
Date: Sat Nov 15 01:25:17 2025 +0800

    初始化按钮

commit 7a96bad8ee2aa984fb42d000e0068526ac3ff6d8
Author: LeoTian <LeoTian163@163.com>
Date: Sat Nov 15 01:04:15 2025 +0800

    修正逻辑，精简代码

commit 8b65329b50b3da064290a2a330b69b7708a0d683 (origin/master)
Author: LeoTian <LeoTian163@163.com>
Date: Wed Sep 17 16:26:49 2025 +0800

    First-Commit, basic frame

E:\Trae Project\npcbot-frontend>
```

后端 git log:

```
commit d1f58aae3829f1ea993c593b1629e219b12f2498 (HEAD -> master)
Author: LeoTian <LeoTian163@163.com>
Date: Sun Nov 16 21:18:10 2025 +0800
```

补一个测试记录

```
commit db73aec487fa1bff996a9dcc2bc8f15de94cf234
Author: LeoTian <LeoTian163@163.com>
Date: Sun Nov 16 00:44:08 2025 +0800
```

接入语言大模型

```
commit 0748d086fa3efbf7e9a433df130d68b3f628db93
Author: LeoTian <LeoTian163@163.com>
Date: Sat Nov 15 21:23:28 2025 +0800
```

更新 docstring

```
commit 83f9675ff73471af2fcc05dd236fdda32ba6d721
Author: LeoTian <LeoTian163@163.com>
Date: Sat Nov 15 20:31:56 2025 +0800
```

修复读取 BUG，请注意 end\_states 的写法，开始构建模块 3 测试脚本

commit 133a3753baf50fd1bb69c842fcff5c758b933c72

Author: LeoTian <LeoTian163@163.com>

Date: Sat Nov 15 18:47:58 2025 +0800

接入自己的 YAML 读取模块

commit 0e08b4c3073c479e7da40229b2ce680866f3a039

Author: LeoTian <LeoTian163@163.com>

Date: Sat Nov 15 18:43:10 2025 +0800

编写并测试 YAML 读取模块

commit 17b3c9dce0c0b1ff31bae4da16de923b14a51f5c

Author: LeoTian <LeoTian163@163.com>

Date: Sat Nov 15 14:14:01 2025 +0800

关闭会话的内存管理

commit 41307b1a1a924b088d50e43139dd1f7816b549bb

Author: LeoTian <LeoTian163@163.com>

Date: Sat Nov 15 01:24:52 2025 +0800

初始化按钮

commit 393b1d4a77a81b05eb326fc0b2092d20a494b712

Author: LeoTian <LeoTian163@163.com>

Date: Sat Nov 15 01:05:14 2025 +0800

修正逻辑，精简代码

commit 1df7edfed627b9f3454cfa7089453c19bae5a89 (origin/master)

Author: LeoTian <LeoTian163@163.com>

Date: Wed Sep 17 16:29:38 2025 +0800

first commitment, basic frame

(END)

## 9 总结与展望

精进了概念设计、模块划分的能力，完整体验一个小项目从设计到实现的流程。

这次任务重点强调关于排错、测试、版本管理的内容，这都是以前任务没有遇到的。

学习到了更多关于前后端开发技术、测试技术、编译原理、理论计算机、语言模型提示词调优等方面的理论和实践知识。

之前任务用到 AI 不会去关注“如何用好”的问题，实现需求即可，这次任务格外关注这个问题。

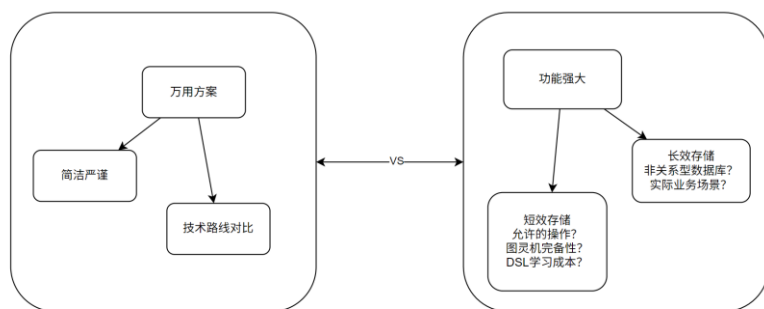
尚未实现的一些设想上：

由于本项目一开始在设计上就想实现一个客服机器人的万用方案，希望“后端把该项目的 app 包复制到根目录，setting.py 注册好，编好 DSL，前端复制到组件区，需要用到的位置导入，前后端启动”之后即可使用，这使得本项目难以设计用户的长效存储方案。

例如，假设确实需要用户首先登录，前后端交互都带着交互令牌，这固然是可行的，这样就可以存储用户关于一些信息的告知。体现在模块 1 上，该脚本可能自带一些特殊符号，其中可能涵盖一个正则表达式去识别用户告知的信息；体现在模块 2 上，自动机的基本模型是可以保持的，但是输出函数可能不简单是个字符串了，而是一系列关于增删改查的操作；体现在模块 4 上，关于大模型的提示词工程会更加复杂，因为既要设计选择类型的消息的返回（“选择疑问句”），又要设计增删改查类型消息的返回（“特殊疑问句”）。

这带来的另一个问题是非关系型数据库的应用。毕竟，DSL 脚本是可以被编程的，而关系型数据库表结构的变化是困难的，这就意味着用户数据的长效存储需要额外的解决方案，最简单的是使用低效的文件存储。

然而，短效的存储方案是可能的。如果采用这个设计，进一步要思考的问题是，允许用户对这些短效存储进行什么操作。如果允许的操作足够复杂，这意味着该 DSL 脚本定义的行为完全有可能和一个图灵机等价（按照理论计算机的观点，达到这个表达能力需要的运算可能并不困难），进而和一套完善的编程语言等价。这附带的问题是 DSL 可能会很复杂，需要大量学习时间。



所以，放到具体实现上，本项目最终采用的模型是简单的，一个 Moore 机。所以本项目在推进时，力求体现出实现这一模型时候的复杂考量（严密说明设计 DSL 的思路、原因、结果，严密说明这个模型的道理，严密说明分析器的方案抉择，严密说明每个技术的选择），并且使得每一个模块划分合理，交互高效，环环相扣。

多功能的强效与严密的美感，这两种路线上的张力是否存在一个解决方案，可能是程序开发者们经常面对的问题。